

Sparse-Latent Koopman Autoencoders for Multibasin Dynamical Systems

Anonymous Authors

Abstract

Koopman autoencoders learn coordinates in which nonlinear dynamics are advanced by a linear latent transition. Multibasin systems make this goal structurally difficult: local linearizations, and sometimes linearizations valid across individual basins, can exist around individual attracting sets, but different basins generally require incompatible finite-dimensional, state-reconstructing linear descriptions. We study sparse-latent Koopman autoencoders, where the encoder is biased to activate only a small subset of latent coordinates. The central hypothesis is that this active support can make the relevant basin or local linear law inspectable for the current state. We test this hypothesis by separating forecasting from interpretability, asking whether supports agree with basin labels in controlled benchmarks, whether they can route local predictors without oracle basin labels, and whether sparse-latent models remain competitive at long horizons. The evidence supports a qualified view. On states far from basin boundaries, sparse representations often form stable supports whose members come from a single basin, suggesting that the encoder recovers part of the attractor organization. However, exact support identity is not a reliable global invariant; it is sensitive to thresholds, basin boundaries, and equivalent latent coordinate choices. For deployment, fixed-size support routing is more robust than thresholded exact matching, and support-conditioned local predictors often improve on applying one global latent transition everywhere. Long-horizon forecasting results show that sparse-latent models remain competitive beyond controlled multibasin benchmarks, with stronger transition structure becoming most useful at the longest horizons. The main conclusion is therefore not that each basin has one exact support, but that induced latent sparsity can reveal support families that help identify the relevant basin or local predictor.

1 Introduction

Learning accurate and interpretable models of nonlinear dynamical systems remains a central problem in machine learning, scientific computing, and control. One particularly attractive direction is to seek a representation in which the nonlinear dynamics can become locally linear, since linear models support efficient prediction, estimation, and control while remaining amenable to classical solving tools such as linear quadratic regulators (LQR) (Kalman, 1960) and model predictive control (MPC) (Mayne et al., 2000; Grüne and Pannek, 2017). Classical linearization theory gives local topological conjugacies near hyperbolic equilibria (Grobman, 1959; Hartman, 1960), and Koopman eigenfunction theory extends related linearizing coordinates to entire basins of attraction for stable equilibria and periodic orbits (Lan and Mezić, 2013; Kvalheim and Revzen, 2021). So indeed, a bridge between nonlinear and linear dynamics can be found theoretically.

Koopman operator theory provides a natural way to do this. Instead of evolving the state in the nonlinear dynamical system directly, Koopman theory studies the evolution of observables under a linear operator acting on a lifted function space (Koopman, 1931; Koopman and Neumann, 1932; Mezić, 2005; Brunton et al., 2022). This viewpoint has inspired a large line of literature on data-driven approximations of Koopman dynamics, such as DMD and eDMD (Schmid, 2010;

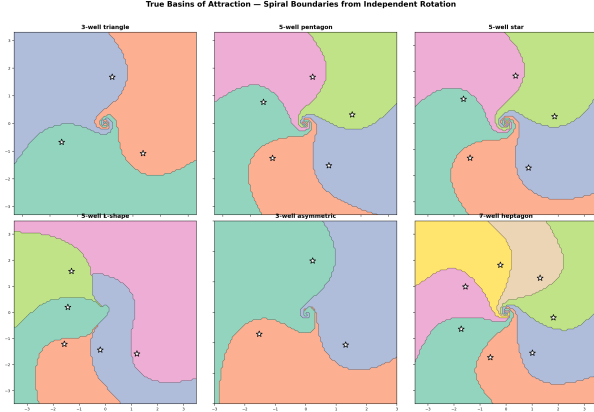
Williams et al., 2015, 2016), and applications of these linear predictors for nonlinear control via MPC (Korda and Mezić, 2018) or LQR (Mamakoukas et al., 2019). More recently, Koopman autoencoders which learn this lift from data have emerged from integrating deep learning with Koopman theory (Takeishi et al., 2017; Lusch et al., 2018; Otto and Rowley, 2019; Azencot et al., 2020; Shi and Meng, 2022; Fathi et al., 2024). Koopman autoencoders learn an encoder which maps a state to a latent representation, a matrix that advances that representation linearly, and a decoder that maps the latent state back to the original state space.

A central challenge of Koopman approximations is that the most interesting nonlinear systems typically contain multiple equilibria or multiple basins of attraction, and theory shows that these systems cannot admit a single finite-dimensional global linearization (Lan and Mezić, 2013; Brunton et al., 2016; Pan and Duraisamy, 2024). For example, a damped mechanical system can settle into different wells; a biological or chemical model can approach different stable equilibria; a controlled system can move between regions with distinct local dynamics. In such multistable systems, it is possible to linearize the dynamics within basins, but the linearizations needed in different basins of attraction are incompatible with each other. Given the theoretical argument, multibasin forecasting with a single global finite-dimensional linearization seems ill-posed; it is more naturally a problem of discovering which local linear description, or which description valid across a basin, applies at the current state.

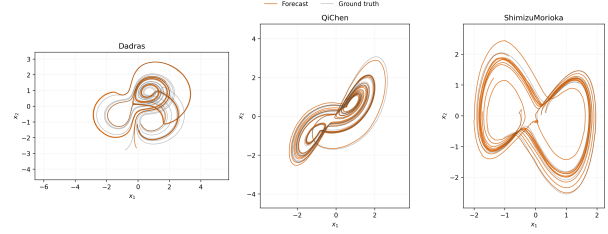
If Koopman models are to serve as a foundation for interpretable multi-regime modeling, then the latent representation of a Koopman autoencoder should do more than forecast well: ideally, it should organize the state space into meaningful local dynamical regimes. Our hypothesis is that sparse coding offers an appealing inductive bias for this purpose. Classical sparse coding and LISTA-style encoders produce piecewise-linear mappings and union-of-subspaces structure in latent space (Olshausen and Field, 1996; Chen et al., 1998; Donoho and Elad, 2003; Gregor and LeCun, 2010). If the latent representation of state produced by a LISTA encoder is sparse, then each state activates only a small subset of latent coordinates. That active subset, which we call a support, is a discrete object that can be inspected and used for routing to correct locally linear dynamics. Then, periodically decoding and re-encoding the state can refresh the quality of the latent embedding over time by re-selecting the correct local dynamics as trajectories move across the state space (Fathi et al., 2024).

This paper tests that hypothesis in a deliberately label-free learning setting. During training and deployment, the model is not told the number of basins, the basin assignment of any trajectory, or which local linear law should be used. Basin labels, when available in controlled benchmarks, are reserved for evaluation. The contribution is an empirical and mechanistic study of whether induced latent sparsity produces support objects that agree with basin labels in evaluation, whether those support objects can route local predictors without oracle labels, and whether the resulting sparse-latent Koopman models remain competitive on long-horizon forecasting tasks.

The paper follows this chain directly. [Section 2](#) formulates multibasin Koopman learning, [section 3](#) explains why sparse supports can identify local dynamical regimes, and [section 4](#) defines the sparse-latent model, support objects, re-encoding rule, and training objective. [Section 5](#) then states the evaluation protocol, including the support-routed prediction diagnostics, and [section 6](#) presents the evidence in the same order as the claim: agreement between supports and basin labels, non-oracle support-routed prediction, support refresh after transfer into a new basin, and long-horizon forecasting.



(a) Representative multibasin benchmark systems.



(b) Long-horizon Dysts forecasting examples at $H20000$.

Figure 1: The paper evaluates two complementary regimes. The procedural multibasin systems provide known basin labels for interpretability diagnostics, while the Dysts systems provide long-horizon forecasting stress tests beyond the controlled low-dimensional setting.

2 Problem formulation

The introduction motivates a multibasin version of Koopman learning. Given sampled trajectories of a nonlinear system, the aim is to learn a state-reconstructing latent representation whose evolution is linear, without being told which basin or local dynamical regime generated each observation. The benchmarks may be defined by continuous-time dynamics, but the learner is only given stored observations. Thus, for each system, we work with the one-step observation map

$$x_{k+1} = F_{\Delta t}(x_k), \quad (1)$$

where $x_k \in \mathbb{R}^{d_x}$ is the state at the k th stored sample, $F_{\Delta t}$ denotes one benchmark observation step, and Δt is the system-specific observation interval. The model does not observe the continuous flow, the vector field, or the internal integration steps used to generate the cached trajectories. Forecasting is therefore defined at the stored sampling cadence: a horizon of H means H applications of $F_{\Delta t}$. Because Δt may differ across benchmark systems, equal values of H need not correspond to equal physical time.

Basins of attraction. Many systems of interest are multistable: different initial conditions can approach different long-run behaviours. A fixed point is a state x^* satisfying $F_{\Delta t}(x^*) = x^*$, equivalently $f(x^*) = 0$ in continuous time. More generally, a trajectory may approach an attractor A , such as a stable equilibrium, limit cycle, torus, or chaotic invariant set. The basin of attraction of A is the set of initial conditions whose trajectories converge to A :

$$\mathcal{B}(A) = \left\{ x_0 \in \mathbb{R}^{d_x} : \text{dist}\left(F_{\Delta t}^k(x_0), A\right) \rightarrow 0 \text{ as } k \rightarrow \infty \right\}. \quad (2)$$

When multiple attractors coexist, their basins partition the state space up to basin boundaries. This creates the regime of interest for this paper. Within one basin, a useful linear description may exist locally or even basin-wide in appropriate coordinates, but another basin may require an incompatible description. Thus, the task is to learn a representation that can support linear prediction while the appropriate linear law may depend on where the current state lies in the multibasin geometry.

Koopman autoencoders. We study this setting with Koopman autoencoders. The model learns an encoder $E_\theta : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$, a decoder $D_\phi : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{d_x}$, and a latent linear transition matrix $K \in \mathbb{R}^{d_z \times d_z}$. Given an initial observation x_k , the corresponding open-loop h -step forecast is

$$\hat{x}_{k+h} = D_\phi \left(K^h E_\theta(x_k) \right). \quad (3)$$

The same matrix K is applied at every step of the rollout, so any state-dependent structure required for prediction must be encoded in the learned representation rather than in a supplied regime label.

Training windows. Training uses windows of adjacent stored observations. For a batch of B windows of rollout length L , $\{x_{b,k}, x_{b,k+1}, \dots, x_{b,k+L}\}_{b=1}^B$, where b indexes windows and ℓ indexes offsets within a window, the model encodes each observed state, rolls out from the initial encoded state, and decodes both reconstructed observations and latent forecasts. The encoded/reconstructed quantities are defined for $\ell = 0, \dots, L$, while rollout/forecast quantities are defined for $\ell = 1, \dots, L$:

$$\begin{aligned} z_{b,k+\ell}^{\text{enc}} &= E_\theta(x_{b,k+\ell}), & z_{b,k+\ell}^{\text{roll}} &= K^\ell z_{b,k}^{\text{enc}}, \\ x_{b,k+\ell}^{\text{rec}} &= D_\phi(z_{b,k+\ell}^{\text{enc}}), & \hat{x}_{b,k+\ell} &= D_\phi(z_{b,k+\ell}^{\text{roll}}). \end{aligned} \quad (4)$$

The training losses described later are built from these reconstructed and rolled-out quantities. Each application of K is interpreted as one stored benchmark step, matching the observation map in [equation \(1\)](#).

Model discovers basins unsupervised. In the intended training and deployment setting, the model is not given basin labels, the number of basins, or trajectory-to-basin assignments. When benchmark basin annotations are available, they are reserved for post-hoc evaluation; they are not used to choose the model, train the encoder or transition, or route predictions.

3 Sparse supports as local-regime indicators

Section 2 ruled out a single global linearization for multibasin systems, yet a Koopman autoencoder applies one shared transition K everywhere. The representation itself must therefore carry whatever distinguishes one local law from another, and we want this carrier to be *inspectable*: continuous coordinates for prediction within a region, and a discrete object naming the active region. Sparse coding supplies both ([Olshausen and Field, 1996](#); [Chen et al., 1998](#); [Donoho and Elad, 2003](#)). For a dictionary $D \in \mathbb{R}^{d_x \times d_z}$ and $\lambda > 0$, the encoder

$$z^\star(x) = \arg \min_z \frac{1}{2} \|x - Dz\|_2^2 + \lambda \|z\|_1 \quad (5)$$

returns coefficients that describe state within a region together with an active set, the *support* $S(x) = \{i : z_i^\star(x) \neq 0\}$, that serves as the discrete indicator. The following standard property of [equation \(5\)](#), a direct consequence of the LASSO KKT conditions, makes the geometric bias precise.

Proposition 1 (Piecewise-affine sparse encoders). *For any D and $\lambda > 0$, the map $x \mapsto z^\star(x)$ defined by [equation \(5\)](#) is continuous on $\mathbb{R}^{d_x}$, and there is a partition of $\mathbb{R}^{d_x}$ into finitely many convex polyhedral cells $\{C_S\}_{S \subseteq \{1, \dots, d_z\}}$ such that on the interior of each C_S the encoder is affine in x with active set exactly S .*

The encoder thus partitions input space into support-indexed cells, with each cell mapped affinely into the $|S|$ -dimensional latent subspace spanned by its active coordinates. A single transition K then advances the latent state along that cell’s active subspace; when x crosses a cell boundary, the support changes and a different submatrix of K carries the dynamics. Multibasin systems—where each basin admits its own linearization but no global one exists—are precisely the regime in which a region-indexed code has predictive content: cell membership encodes *which* local law applies, while the continuous coefficients evolve under it.

This is a structural argument, not a guarantee. [Proposition 1](#) fixes D , but in our model the dictionary, shrinkage scale, and K are co-trained against the prediction objective [equation \(16\)](#), and only training pressure can drive cell boundaries to coincide with basin boundaries. Whether it does so is an empirical question, and the argument decomposes into two claims that we evaluate separately. Statically, the support of an encoded state should reduce uncertainty about the basin. Dynamically, the same support should select *which* active subspace of K , or which fitted local linear law, advances prediction. The first does not imply the second: an encoder could mark basin identity with some always-active coordinates while routing prediction through unrelated ones. [Section 5](#) reports support–basin entropy for the static claim and a wrong-support ablation plus support-conditioned routing for the dynamical one.

4 Method

4.1 Model class

Can a Koopman autoencoder trained without basin supervision learn a discrete, inspectable object that identifies the relevant basin or local predictive regime? We investigate the following chain

$$\text{sparse latent code} \implies \text{support agrees with basin labels} \implies \text{support selects local prediction.} \quad (6)$$

The model and objective induce the sparse code. The two arrows are evaluated after training: first, by asking whether the support agrees with basin labels that are withheld during training; second, by asking whether the same support can select useful local predictors without receiving a basin label at test time.

The underlying predictor is a Koopman autoencoder with encoder E_θ , decoder D_ϕ , and latent transition K . Using a linear decoder, the maps are

$$z_t = E_\theta(x_t) \in \mathbb{R}^{d_z}, \quad D_\phi(z) = Dz, \quad \hat{z}_{t+1} = Kz_t, \quad \hat{x}_{t+1} = D_\phi(\hat{z}_{t+1}). \quad (7)$$

An active coordinate can therefore be read as selecting a learned reconstruction direction, while the latent state is advanced by the same linear transition between re-encoding events. Sparsity is a property of the learned representation; it is not an assumption that the physical state or the underlying dynamical system is sparse.

Sparse encoder. Our main sparse encoder uses LISTA-style amortized sparse inference ([Gregor and LeCun, 2010](#)). The encoder first computes a dense pre-code $c_t = f_\theta(x_t)$ and then applies learned shrinkage refinements, for $q = 0, \dots, Q - 1$:

$$u_t^{(0)} = \text{shrink}(c_t, \tau), \quad u_t^{(q+1)} = \text{shrink}(Su_t^{(q)} + c_t, \tau), \quad z_t = u_t^{(Q)}. \quad (8)$$

Here $\text{shrink}(a, \tau) = \text{sign}(a) \max(|a| - \tau, 0)$ is the standard elementwise soft-thresholding map, S is learned, and τ sets the shrinkage scale. The thresholding operation gives the encoder an explicit mechanism for selecting active coordinates, while the learned pre-code and refinement weights allow the selection rule to adapt to the prediction objective.

Transition families. We vary the structure of K separately from the encoder, which separates the effect of induced sparsity from the effect of the latent transition. The dense transition leaves K unrestricted. The block-diagonal transition partitions the latent coordinates into M groups and constrains

$$K_{\text{block}} = \text{blkdiag}(K_1, \dots, K_M), \quad (9)$$

so each group evolves internally. The soft-block transition keeps K dense but penalizes entries that couple different groups,

$$\mathcal{L}_{\text{offblock}} = \sum_{i,j: g(i) \neq g(j)} K_{ij}^2, \quad (10)$$

where $g(i)$ denotes the group containing coordinate i . These groups are architectural coordinates, not basin labels. All basin comparisons are performed only after training.

4.2 Support objects

The central object in the paper is the active set induced by an encoded state. Given $z = E_\theta(x)$, a support rule converts z into a set of active coordinates. The main displays use two support rules, each with a different role:

- **Absolute-threshold support.** $S_{\text{abs}}(x) = \{i : |z_i| > 10^{-3}\}$. This is the strictest test of whether a single active set dominates far from basin boundaries.
- **Fixed-size top-8 support.** $S_{\text{top8}}(x) = \text{top}_8(|z|)$, the indices of the eight largest-magnitude coordinates. This is the most deployment-like support because every state receives a route key of the same size.

A third scale-relative rule, $S_{\text{rel}}(x) = \{i : |z_i| > 0.1 \max_j |z_j|\}$, is used only in appendix sensitivity diagnostics for centered local laws.

We distinguish exact supports from support families. An *exact support* is the set returned by a support rule. It is a stringent object: if one exact support identifies one basin, the claim is easy to inspect. A *support family* groups nearby exact supports by Jaccard overlap, using threshold 0.5 against the family prototype. This allows one basin or local regime to be represented by several closely related masks, which is expected near basin boundaries or when a weak coordinate crosses an activity threshold. Family-routed prediction assigns a test-time exact support to a previously fitted family only if it matches closely enough; otherwise the prediction falls back to the global transition.

4.3 Periodic re-encoding

If a support indicates the currently relevant local dynamics, then a long forecast should be allowed to update the support as the predicted state moves. We therefore evaluate periodic re-encoding, following Fathi et al. (2024). Starting from $z_k = E_\theta(x_k)$ and a re-encoding period m , the model advances for m Koopman steps, decodes the predicted state, and encodes that prediction again:

$$\tilde{z}_{k+m} = K^m z_k, \quad \tilde{x}_{k+m} = D_\phi(\tilde{z}_{k+m}), \quad z_{k+m} = E_\theta(\tilde{x}_{k+m}). \quad (11)$$

The same procedure is repeated over the rollout horizon. Each rollout uses only its own predicted state; it does not use the true future state, a basin label, or an oracle for transitions between basins.

4.4 Support-conditioned rollouts

We also define two support-conditioned one-step rules used in the routing diagnostics. Let c be the support object inferred from the current predicted latent state, let m_c be its binary mask, and let

$\bar{z}_c$ be the mean latent state associated with that support object on a disjoint fitting set. The first rule masks the centered input before applying the learned global transition:

$$\hat{z}_{t+1} = \bar{z}_c + K(m_c \odot (z_t - \bar{z}_c)). \quad (12)$$

The second rule fits a centered local linear predictor for each support object:

$$A_c = \arg \min_A \sum_{t \in \mathcal{I}_c} \|(z_{t+1} - \bar{z}_c) - A(z_t - \bar{z}_c)\|_2^2 + \eta \|A\|_F^2, \quad (13)$$

and predicts $\bar{z}_c + A_c(z_t - \bar{z}_c)$. These rules test whether the learned support can replace a supplied regime label when selecting a local predictor.

4.5 Training objective

Training uses only adjacent observation windows. For a batch of B windows with prediction horizon L , the model produces reconstructed future states $x_{b,k+\ell}^{\text{rec}}$, latent rollouts $z_{b,k+\ell}^{\text{roll}}$, encoded future latents $z_{b,k+\ell}^{\text{enc}}$, and decoded rollout predictions $\hat{x}_{b,k+\ell}$ as defined in [equation \(4\)](#). The rollout is seeded by the initial state, and the following sums are over offsets $\ell = 1, \dots, L$:

$$\bar{\mathcal{L}}_{\text{pred}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|\hat{x}_{b,k+\ell} - x_{b,k+\ell}\|_2, \quad \bar{\mathcal{L}}_{\text{rec}} = \frac{1}{BL\sqrt{d_x}} \sum_{b,\ell} \|x_{b,k+\ell}^{\text{rec}} - x_{b,k+\ell}\|_2, \quad (14)$$

$$\bar{\mathcal{L}}_{\text{lin}} = \frac{1}{BL} \sum_{b,\ell} \|z_{b,k+\ell}^{\text{roll}} - z_{b,k+\ell}^{\text{enc}}\|_2, \quad \bar{\mathcal{L}}_{\text{sp}} = \frac{1}{BL} \sum_{b,\ell} \|z_{b,k+\ell}^{\text{roll}}\|_1. \quad (15)$$

The observation-space terms are normalized by $\sqrt{d_x}$ so that their scale is comparable across systems of different state dimension. The latent terms are left in their native scale because the latent dimension is fixed within each comparison.

The total objective is

$$\mathcal{L} = \frac{1}{L} (\lambda_{\text{pred}} \bar{\mathcal{L}}_{\text{pred}} + \lambda_{\text{rec}} \bar{\mathcal{L}}_{\text{rec}} + \lambda_{\text{lin}} \bar{\mathcal{L}}_{\text{lin}} + \lambda_{\text{sp}} \bar{\mathcal{L}}_{\text{sp}}) + \mathcal{L}_{\text{struct}}. \quad (16)$$

Here $\mathcal{L}_{\text{struct}} = 0$ for the dense and block-diagonal transition variants. For the soft-block transition, $\mathcal{L}_{\text{struct}} = \lambda_{\text{off}} \mathcal{L}_{\text{offblock}}$. The prediction and latent-linearity terms make the representation useful for Koopman rollout, the reconstruction term keeps the code tied to the observed state, and the sparsity term encourages selective activation. No loss term uses basin labels, basin counts, or trajectory-to-basin assignments.

Compared with the minimal Koopman autoencoder objective in [equation \(26\)](#), this rollout objective adds decoded multi-step prediction, latent consistency over stored observation windows, explicit sparsity pressure, and any transition-structure penalty. These additions are needed because the paper is not only asking whether one-step latent features can be made approximately linear; it is asking whether the learned support remains useful for finite-horizon prediction and can be inspected as a label-free local-regime object.

5 Experimental procedure

The experiments follow the chain in [equation \(6\)](#). We first ask whether sparse supports agree with basin labels that were withheld during training. We then ask whether those supports are functionally used by the predictor, whether they can route local forecasts without an oracle label, whether periodic re-encoding keeps the selected support current after basin transfer, and finally whether the same sparse-latent models remain competitive in a longer-horizon forecasting regime without basin labels.

Benchmarks and training. [TODO: update this once we have 15 seeds; stronger statistical significance tests. Note that we reduce dysts to 12 systems only on our re-run.] The controlled benchmark contains 17 procedurally generated two-dimensional multibasin systems, spanning multiwell potentials, gated local-linear systems, polygonal arrangements with 3–10 attractors, checkerboard potentials, depth-cascade variants, and transition-rich layouts. Basin labels are available only for evaluation: they are not used for training, model selection, support extraction, or routing. All multibasin models are trained from a distribution that deliberately includes many initial conditions near basin boundaries, so the learned representation must handle both unambiguous basin interiors and boundary-adjacent states. We compare the five model rows shown in [table 1](#): two LISTA sparse encoders with structured latent transitions, two sparse MLP controls with dense or block-diagonal transitions, and a Dense MLP control with no sparsity penalty. Each model–system cell is trained for ten random seeds; paired analyses use the common completed seeds. All rows use length-8 training windows and 200,000 optimization steps under the objective in [equation \(16\)](#). We use a subset of 12 chaotic systems from the Dysts benchmark ([Gilpin, 2021, 2023](#)) for forecasting only, since it does not supply basin labels.

Forecasting and statistics. [TODO: Check that we actually have the longer horizon evals from dysts] Forecasting is evaluated on held-out trajectories at the stored observation cadence. On the multibasin benchmark we report raw mean squared error at $H \in \{100, 500, 1000\}$; on Dysts we report $H \in \{5000, 10000, 20000, 30000, 40000, 50000, 60000\}$. The headline forecasting score is the best value over a fixed periodic re-encoding grid. This procedure uses the model’s own predicted state for every re-encoding event and never uses the true future state or a basin label. Point estimates are interquartile means over seeds within each system, followed by an interquartile mean across systems. For each candidate row and horizon, we compare against the Dense MLP baseline within each system using paired $\log_{10}$ errors over seeds, apply a one-sided paired Wilcoxon signed-rank test, and Holm-correct the resulting p -values across systems at $\alpha = 0.05$. The reported K/N values count systems that pass this correction.

Support diagnostics. Support–basin agreement is evaluated on held-out states. The main alignment table uses the absolute-threshold support on the deep-state subset, defined as the top quartile of states by distance from a basin boundary. This focuses the identifiability test on states well inside basins, where a basin-local description should be most stable. We report the mean active support size, basin uncertainty given an exact support, and basin uncertainty given a support family. These quantities must be interpreted together: a low basin uncertainty is meaningful only if the active set is also small enough to be an inspectable object.

We test functional use of the support with a wrong-support ablation. For a held-out deep-state example, the model’s inferred support is replaced by a canonical support from another basin and then held fixed for the rollout. The reported ratio is the wrong-support error divided by the ordinary forecast error. Ratios near one indicate that the swap did not matter; large ratios indicate that the predictor depends on the selected support. This ablation is undefined when the deep-state subset contains only one basin, so those systems are excluded from that denominator.

Routing and refresh. The non-oracle routing experiment freezes the trained representation and fits the support-conditioned predictors in [equations \(12\) and \(13\)](#) on a disjoint fitting set. During evaluation, the route at each step is the model’s own predicted top-8 support; if the route has too few fitted transitions, the rollout falls back to the global latent transition. The metric is the routed/global error ratio for the same trained model, so values below one measure the benefit of

routing without changing the representation or using a basin label. The routing controls in the appendix replace learned supports with oracle basin partitions, random partitions of matched count, or latent clusters.

The refresh experiment evaluates support selection after controlled transfer from one basin into another. It compares a rollout that keeps using the source-basin support after entry into the target basin with a rollout that periodically decodes, re-encodes, and re-selects its support. Basin labels are used only to construct and score these transfers. The reported quantities are the fraction of post-entry steps routed through the target class and the refreshed/previous forecast-error ratio; because this packet uses one training seed, the paired test unit is the controlled transfer pair rather than the training seed. Additional procedural details are collected in [section B](#).

6 Results

The results follow the protocol order of [section 5](#): support–basin agreement, functional dependence on the support, non-oracle routing, support refresh under basin transfer, and long-horizon forecasting. The first four claims are evaluated on the controlled multibasin benchmark, where basin labels are withheld during training and used only for evaluation; the final claim extends the comparison to a chaotic regime in which the same models are tested as forecasters.

Table 1: [\[TODO: Update this table with updated statistical significance tests once we have 15 total seeds for each setup and we have also sparse-MLP and sparse-MLP-BD setups. Should move the entropy and wrong support columns to the left since they are the key interpretability results, and move horizon forecasting to the right columns. Might also want to check LISTA-SB at \$d_z = 256\$ once it finishes.\]](#) Multibasin forecasting and support–basin agreement on the 17-system benchmark. Each cell shows the metric followed by $[K/N]$, the number of systems on which the row improves over the Dense MLP baseline after Holm correction at $\alpha=0.05$ (per-system one-sided paired Wilcoxon over 10 seeds; [section 5](#)). Forecasting columns are cross-system IQM of per-system IQM-over-seeds in raw MSE. The wrong-support ablation columns use a $K/13$ denominator because the diagnostic is undefined on 4 systems whose deep-state subset is single-basin. Underlined entries are best in column.

Model	$H(B S) \downarrow$	$H(B F) \downarrow$	wr. $h=1 \uparrow$	wr. $h=20 \uparrow$	$ S \downarrow$	H100 $\downarrow$	H1000 $\downarrow$
LISTA-BD	0	<u>0</u> [13/17]	8.1×10^3 [13/13]	16.4 [10/13]	101 [17/17]	0.0175 [14/17]	0.0699 [15/17]
LISTA-SB	0	0.055 [11/17]	3.0×10^3 [13/13]	8.22 [7/13]	<u>27</u> [17/17]	0.0255 [15/17]	0.0708 [15/17]
Sparse MLP, BD	0	0.00043 [12/17]	6.3×10^3 [13/13]	35.2 [12/13]	106 [17/17]	<u>0.0163</u> [15/17]	<u>0.0452</u> [17/17]
Sparse MLP	0	<u>0</u> [13/17]	6.5×10^3 [13/13]	<u>35.3</u> [12/13]	106 [17/17]	0.0183 [14/17]	0.0465 [17/17]
Dense MLP, no shrink	0.0012	0.9 [baseline]	8.68 [baseline]	1.35 [baseline]	254 [baseline]	0.665 [baseline]	3.07 [baseline]

Sparse supports identify basin membership. On states far from basin boundaries, every sparse model attains near-zero basin uncertainty given the inferred support ([table 1](#)). The dense baseline reaches comparable uncertainty, but only by activating nearly every latent coordinate, so the discrete object it returns is not interpretable as an indicator of basin membership. The sparse encoders instead concentrate basin information into a small number of support families, while the dense baseline degenerates to a single family that pools all systems together—the sharpest interpretability gap in the table. Per-system testing rejects the support-size claim on every benchmark system for each sparse model and the family-entropy claim on a clear majority, and the per-(system, seed) distributions in [figure 2](#) confirm that the gap is consistent rather than driven by outliers.

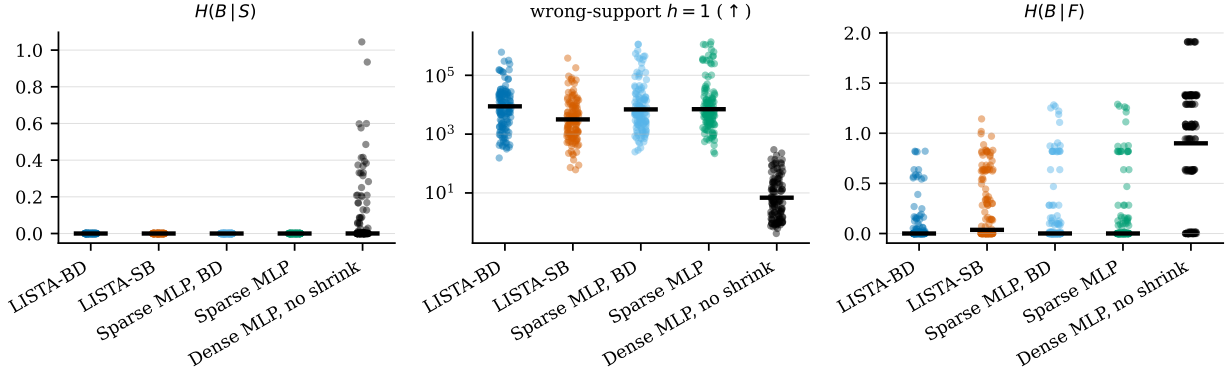


Figure 2: [TODO: might need to regenerate this once we have more (15) seeds.] Per-(system, seed) basin certainty, functional dependence on the support, and family-level basin certainty on the deep-state subset (S_{abs}). Each dot is one (system, seed) pair; the black bar is the cross-system IQM. Lower is better for $H(B|S)$ and $H(B|F)$; *higher* is better for the wrong-support ratio (replacing the inferred support with a wrong-basin canonical support and holding it fixed for $h=1$ rollout: a large ratio means the swap breaks prediction, so the model genuinely uses the support). Every panel corresponds to a column reported in table 1.

Prediction functionally depends on the support. Support-basin agreement does not by itself imply that the support drives prediction. The wrong-support ablation in table 1 replaces the inferred support with a canonical support drawn from a different basin and holds the swap fixed throughout the rollout. The forecasts of every sparse model degrade by several orders of magnitude at one-step horizon and remain markedly worse twenty steps out, while the dense baseline barely registers the swap. Per-system testing rejects on every testable system at the one-step horizon and on the majority at twenty steps. The predictor therefore reads the support: the support is not a passive record of which coordinates happened to be active.

Supports route useful local predictors without oracle basin labels. Table 2 uses each model’s own predicted fixed-size support as the routing key at every step and compares its routed rollout against its own dense rollout, isolating the value of routing inside an otherwise identical learned representation. Both LISTA variants reduce error on a majority of systems on the deep-state subset; the variant with the soft-block transition gives the strongest effect when routing on exact supports, while the block-diagonal variant produces sharper support families and the strongest family-level routing. The dense baseline is flat under either rule, consistent with the absence of a coherent family structure noted above. Because every state receives a route key of the same size, the gain cannot be attributed to silently dropping difficult states. The falsification controls calibrate this finding: oracle basin partitions improve local prediction for every model family, but neither random partitions matched for class count nor latent clusters reproduce the routed-versus-global ratios, so the effect is specific to the partition the sparse encoder learns rather than a generic benefit of subdividing state space.

Refreshing the support with periodic re-encoding tracks basin transfer. The routing test starts inside a single basin. For deployment over long horizons, the support must remain current as a trajectory crosses basin boundaries. Table 3 compares a stale source-basin support held throughout the rollout against a support refreshed by the periodic re-encoding rule of equation (11) after a

Table 2: [TODO: Update this table with updated statistical significance tests once we have 15 total seeds for each setup and we have also sparse-MLP and sparse-MLP-BD setups. Might also want to check H1000 as there might be a larger gap for our benefit. Might also want to check LISTA-SB at $d_z = 256$.] Non-oracle support-routed forecasting at $H100$ on the multibasin benchmark using top-8 supports. The route at every step is the model’s own predicted support; basin labels are not used. Each cell is the cross-rollout IQM of the routed/same-model-global MSE ratio followed by $[K/17]$, where K is the number of systems whose paired one-sided Wilcoxon test clears Holm correction at $\alpha=0.05$. Lower ratios are better; values below one mean that routing helped. Bold entries are best within each subset and route column.

Model	Subset	Support-gated K	Centered support-local	Centered family-local
LISTA-SB	all	0.47 [12/17]	0.45 [11/17]	0.24 [6/17]
LISTA-SB	deep	0.38 [14/17]	0.36 [15/17]	0.061 [11/17]
LISTA-BD	all	0.92 [0/17]	0.92 [1/17]	0.10 [11/17]
LISTA-BD	deep	0.93 [0/17]	0.93 [3/17]	0.032 [12/17]
Dense MLP	all	1.00 [0/17]	1.00 [0/17]	> 290 [0/17]
Dense MLP	deep	0.99 [0/17]	0.99 [0/17]	> 220 [0/17]

controlled entry into a target basin. Refreshed rollouts route through the target basin on most post-entry steps and reduce forecast error by two to four orders of magnitude relative to the stale rollout, with per-system tests passing on nearly every transfer-pair system. The improvement is stable across the re-encoding periods we evaluated, indicating that the mechanism is not sensitive to a particular schedule. Periodic re-encoding therefore supplies a label-free way of keeping the local description aligned with the current region of state space, completing the deployment-time picture begun by the routing test.

Table 3: Support refresh after controlled basin transfer (top-8 supports, 356–396 post-entry rollouts per cell). *Route-target*: fraction of post-entry steps that route through the target’s local class. *MSE ratio (IQM)*: cross-rollout interquartile mean of refreshed-support/previous-support forecast MSE, with $[Q25, Q75]$ shown for context. $K/12$: number of systems whose paired one-sided Wilcoxon test on transfer-pair $\log_{10}(\text{refreshed}/\text{previous})$ ratios clears Holm correction at $\alpha=0.05$.

Model	Period	Route-target	Fallback	MSE ratio vs prev. support: IQM $[Q25, Q75]$	$K/12$
LISTA-SB	1	0.86 (0.30)	0.14	2.4×10^{-4} [9.5×10^{-5} , 4.8×10^{-4}]	11/12
LISTA-SB	10	0.89 (0.19)	0.11	3.3×10^{-4} [1.3×10^{-4} , 6.5×10^{-4}]	11/12
LISTA-BD	1	0.74 (0.36)	0.26	0.28 [0.07, 0.64]	10/12
LISTA-BD	10	0.82 (0.26)	0.18	0.053 [0.008, 0.18]	10/12

Sparse models improve long-horizon forecasting of multibasin systems and chaotic flows.

The interpretability gains come at no forecasting cost. On the controlled multibasin benchmark, every sparse model reduces cross-system MSE by roughly an order of magnitude relative to the dense baseline at the longest evaluated horizon, with per-system improvements on a strong majority of the 17 systems across horizons from 100 to 1000 stored steps (table 1, figures 3 and 5); the dense baseline carries a substantially heavier upper tail at every horizon. The two sparse-MLP controls track the LISTA variants closely, indicating that the active ingredient is induced latent sparsity rather than the LISTA architecture per se. On the 12 chaotic Dysts systems (table 4 and figure 4), the

absolute gap from the dense baseline is smaller, but a coherent within-row pattern emerges: a dense latent transition with sparse encoding leads at shorter horizons, while a block-diagonal transition combined with stronger sparsity wins at the longest horizons, consistent with more constrained transition structure paying off as accumulated drift dominates. Because Dysts trajectories carry no basin labels, this evidence speaks only to forecasting and does not bear on support–basin alignment. [TODO: Refresh this paragraph and table 4 once the new 15-seed batch on the 12-system Dysts shortlist completes; the qualitative ordering is expected to persist while the per-system K/N counts firm up.]

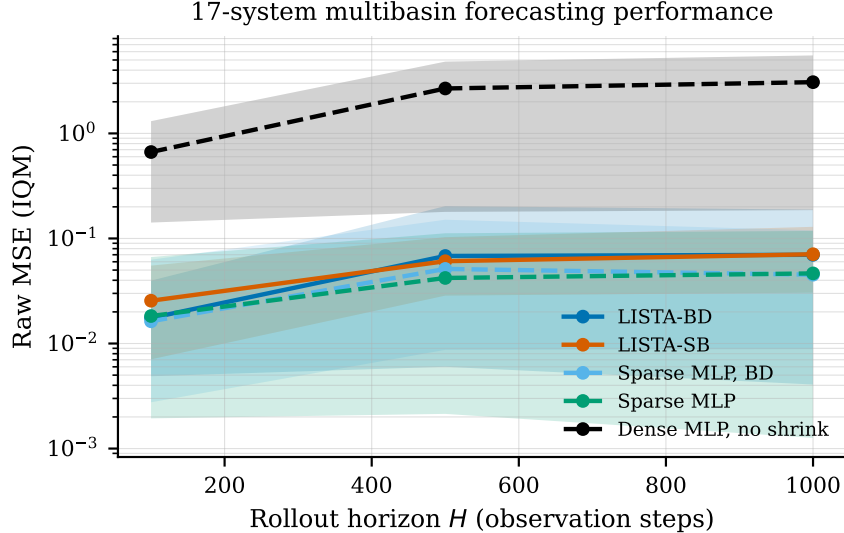


Figure 3: [TODO: Put this side-by-side with the dysts forecasting performance once finished; fig:dysts_long_horizon should be integrated into this. So 2 subfigures in a row.] Multibasin forecasting horizon curves. Lines are cross-system IQM of per-system IQM-over-seeds; bands are the 25–75 system percentile range. The Dense MLP degrades by more than an order of magnitude at every horizon, while the four sparse rows remain close in aggregate error.

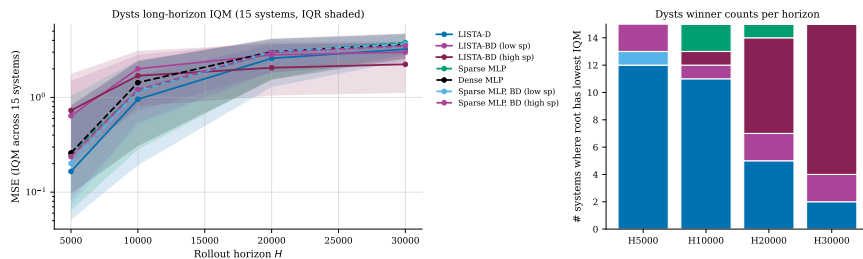


Figure 4: Dysts long-horizon forecasting curves. Lines summarize the same held-out Dysts evaluation packet as table 4; the benchmark is used only for forecasting because basin labels are not available.

7 Related work

Koopman learning. Koopman operator theory represents nonlinear dynamics through a linear operator on observables (Koopman, 1931; Mezić, 2005). EDMD approximates this operator with a

Table 4: [TODO: Wait for the 15 new seeds of 5 model recipes on the 12 system shortlist to finish. Once done, regenerate this table with the appropriate statistical significance tests.] Long-horizon Dysts forecasting. Each cell is the cross-system IQM of per-system IQM-over-seeds (system standard deviation in parentheses), followed by $[K/15]$, the number of systems whose within-system one-sided paired Wilcoxon test against the Dense MLP clears Holm correction at $\alpha=0.05$. LISTA-D is the dense-transition LISTA model; LISTA-BD is the block-diagonal LISTA model at two sparsity strengths. Cross-system $\log_{10}$ standard deviations are in [section G](#).

Model	H5000	H10000	H20000	H30000
LISTA-D	0.166 (0.57) [1/15]	0.957 (1.31) [1/15]	2.585 (1.92) [1/15]	3.223 (2.02) [1/15]
LISTA-BD (low sp)	0.640 (1.13) [0/15]	2.003 (1.62) [0/15]	2.816 (1.73) [0/15]	3.001 (1.62) [0/15]
LISTA-BD (high sp)	0.729 (1.07) [0/15]	1.694 (1.38) [0/15]	2.057 (1.60) [1/15]	2.235 (1.63) [1/15]
Sparse MLP	0.256 (0.71) [0/15]	1.168 (1.43) [1/15]	2.990 (1.60) [0/15]	3.786 (1.44) [0/15]
Sparse MLP, BD (low sp)	0.201 (0.71) [0/15]	1.171 (1.37) [0/15]	3.037 (1.78) [0/15]	3.669 (1.78) [0/15]
Sparse MLP, BD (high sp)	0.237 (0.82) [0/15]	1.211 (1.44) [0/15]	2.991 (1.80) [0/15]	3.504 (1.80) [0/15]
Dense MLP	0.259 (0.62)	1.433 (1.35)	3.035 (2.10)	3.618 (2.05)

chosen dictionary ([Williams et al., 2015](#)). Neural Koopman methods learn the dictionary or invariant subspace from data ([Takeishi et al., 2017](#); [Lusch et al., 2018](#); [Otto and Rowley, 2019](#)). Consistent and course-corrected Koopman autoencoders improve rollout behavior through additional losses and correction mechanisms ([Azencot et al., 2020](#); [Fathi et al., 2024](#)). Our contribution is not a new claim that neural Koopman autoencoders can forecast. It is an empirical study of whether sparse latent supports carry information about the relevant basin or local dynamics.

Multibasin Koopman structure. Multiple invariant sets complicate naive interpretations of a single global finite-dimensional Koopman representation. Prior theory shows that global lifting and reconstruction depend on invariant-set structure, spectra, and symmetry ([Lan and Mezić, 2013](#); [Pan and Duraisamy, 2024](#)). This motivates our use of support families and top- k supports rather than a universal exact-support claim.

Sparse coding and LISTA. Sparse coding is a classical representation principle in which signals are reconstructed from a small number of active atoms ([Olshausen and Field, 1996](#)). LISTA learns fast amortized sparse inference ([Gregor and LeCun, 2010](#)). We use this sparse-inference bias inside a dynamical model and evaluate whether the resulting active atoms form support objects that agree with benchmark basin labels.

Switching and local linear models. Switching state-space models, hybrid systems, and local Koopman methods explicitly introduce modes or local operators ([Linderman et al., 2017](#); [Torrisi and Bemporad, 2004](#); [Mamakoukas et al., 2019](#)). Our model differs because it does not train with known basin labels or a known basin count. The switching behavior, when it appears, is implicit in the learned sparse support.

8 Limitations and conclusion

The results support a qualified, falsifiable claim. Sparse-latent KAEs can learn support objects that agree with basin labels on states far from basin boundaries, and fixed-size top-8 supports can route

useful local forecasts without oracle labels. Exact support uniqueness is not guaranteed globally. It is strongest away from separatrices and should not be used as a brittle thresholded deployment rule.

The evidence also separates sparsity from LISTA-specificity. Sparse MLP controls are strong, so the paper should not claim that LISTA is the only route to the observed behavior. The safer conclusion is that induced sparsity is a useful inductive bias for Koopman representations whose active supports agree with basin labels, with LISTA providing one structured implementation of that bias.

Several limitations remain. The current true-geometry evidence is mixed, so we do not claim recovery of true Jacobian eigendirections. Controlled transfer between basins is positive for dense sparse-latent top-8 supports and for support families, but it is not uniformly positive for every transition structure or every exact support definition. The benchmark systems used for support-label comparisons are low-dimensional, and higher-dimensional systems may require additional structure or weaker support-family interpretations.

Overall, sparse support objects provide a useful bridge between forecasting and interpretability. They expose discrete structure related to basin membership and local dynamics inside a Koopman autoencoder, and the current evidence shows that this structure can agree with basin labels and route local predictive laws. The most robust paper claim is not that each basin has one exact support, but that induced sparse support objects can reveal and exploit local dynamical structure when evaluated with appropriate support definitions and falsification diagnostics.

A Additional background

A.1 Koopman operators and finite-dimensional approximations

Consider a discrete-time nonlinear dynamical system

$$x_{t+1} = f(x_t), \quad x_t \in \mathcal{X} \subset \mathbb{R}^{d_x}. \quad (17)$$

Koopman operator theory studies the evolution of observables rather than the evolution of the state directly. An observable is a measurement function $h : \mathcal{X} \rightarrow \mathbb{R}$, and the Koopman operator $\mathcal{K}$ acts by composition:

$$(\mathcal{K}h)(x) = h(f(x)). \quad (18)$$

The map f may be nonlinear, but $\mathcal{K}$ is linear on the function space of observables. If a finite vector of observables

$$\Phi(x) = [h_1(x), \dots, h_{d_z}(x)]^\top \quad (19)$$

spans a Koopman-invariant subspace, then there is a matrix $K_\Phi \in \mathbb{R}^{d_z \times d_z}$ such that

$$\Phi(x_{t+1}) = \Phi(f(x_t)) = \mathcal{K}\Phi(x_t) = K_\Phi \Phi(x_t). \quad (20)$$

In that case the nonlinear dynamics are exactly linear in the observable coordinates $z_t = \Phi(x_t)$. Learning Koopman models can therefore be viewed as searching for observables whose span is invariant enough to support useful finite-dimensional linear prediction.

In practice, the infinite-dimensional operator is approximated from data. Given snapshot pairs $\{(x_t, x_{t+1})\}$ and a chosen feature map Φ , dynamic mode decomposition and extended dynamic mode decomposition estimate a finite transition by least squares (Schmid, 2010; Rowley et al., 2009; Tu et al., 2014; Williams et al., 2015, 2016):

$$K_\Phi = \arg \min_{\hat{K}} \sum_t \left\| \Phi(x_{t+1}) - \hat{K} \Phi(x_t) \right\|_2^2. \quad (21)$$

DMD is the special case $\Phi(x) = x$, so it fits a linear state-space surrogate without a learned nonlinear lift. EDMD extends this by choosing a richer feature dictionary. Koopman autoencoders replace the fixed dictionary with a learned encoder and decoder, which is the setting used in this paper.

The same lifted-linear viewpoint is also useful for controlled systems, even though the experiments in this paper are autonomous. For dynamics

$$x_{t+1} = f(x_t, u_t), \quad (22)$$

Koopman control models often learn observables $z = \psi(x)$ and matrices A, B, C such that

$$z_{t+1} \approx Az_t + Bu_t, \quad x_t \approx Cz_t. \quad (23)$$

The resulting predictor can be combined with linear control tools such as LQR or MPC while the nonlinearity is absorbed into the lifting map and decoder (Kalman, 1960; Korda and Mezić, 2018; Mayne et al., 2000). Linear latent dynamics also simplify system identification and long-horizon rollout because the same finite matrix can be fitted, analyzed, and iterated directly. This paper does not evaluate control policies, but this motivation is important for interpretation: if a support can identify the currently relevant local linear law, then it is not merely a post-hoc label, but a possible routing object for prediction, planning, or control.

A.2 Why multibasin systems require local structure

The theoretical obstruction in multibasin systems is not the absence of local linear descriptions. Near appropriate attracting sets, classical linearization and Koopman eigenfunction constructions can produce useful local or basin-restricted coordinates. The obstruction is instead global: a single finite-dimensional, state-reconstructing Koopman-invariant subspace is highly restrictive when multiple invariant sets coexist. In particular, results summarized by Lan and Mezić (2013) and Brunton et al. (2016) show that exact finite-dimensional invariant subspaces containing the state coordinates are compatible only with limited global attractor structure.

Equivalently, an attractor A induces a basin

$$\mathcal{B}(A) = \{x \in \mathcal{X} : \text{dist}(f^t(x), A) \rightarrow 0 \text{ as } t \rightarrow \infty\}, \quad (24)$$

and different basins can require different local coordinates or different linear predictive laws. Basin boundaries are therefore precisely the regions where a representation that was valid for one local chart may need to switch to another.

For the present paper, this motivates a cautious formulation. A learned finite-dimensional Koopman representation on a multibasin system should not be expected to be one exact global linearization that reconstructs every basin simultaneously. It must either approximate across incompatible regions, or it must contain some implicit information about which local description is active. Sparse supports are useful because they make one such piece of information discrete and inspectable: the support can identify a basin, basin part, or local predictive chart while the coefficient values carry within-chart state information.

A.3 Koopman autoencoders and periodic re-encoding

A standard Koopman autoencoder learns an encoder $E_\theta : \mathbb{R}^{d_x} \rightarrow \mathbb{R}^{d_z}$, decoder $D_\phi : \mathbb{R}^{d_z} \rightarrow \mathbb{R}^{d_x}$, and latent transition K so that

$$z_t = E_\theta(x_t), \quad z_{t+1} \approx Kz_t, \quad x_t \approx D_\phi(z_t). \quad (25)$$

A minimal one-step objective has the form

$$\min_{E_\theta, D_\phi, K} \sum_t \|x_t - D_\phi(E_\theta(x_t))\|_2^2 + \lambda_{\text{lin}} \|E_\theta(x_{t+1}) - KE_\theta(x_t)\|_2^2. \quad (26)$$

This objective is useful as a reference point, but it is not sufficient for the present study. It does not directly train decoded multi-step rollouts, does not impose a sparse support object, and does not test whether the support itself selects a useful local predictive rule. The training objective in [section 4.5](#) therefore keeps the autoencoding and latent-linearity terms but adds decoded rollout prediction over windows, sparsity on the rolled latent states, and optional transition-structure penalties.

After training, a one-shot Koopman rollout encodes once, advances by repeated multiplication with K , and decodes at the requested horizon. Periodic re-encoding instead interrupts the latent rollout after a fixed number of stored observation steps. For period r ,

$$\tilde{z}_{t+r} = K^r z_t, \quad \tilde{x}_{t+r} = D_\phi(\tilde{z}_{t+r}), \quad z_{t+r} = E_\theta(\tilde{x}_{t+r}). \quad (27)$$

This is the same mechanism defined in [equation \(11\)](#). It uses only the model’s own predicted state and is therefore not a teacher-forced correction. The point is to refresh the latent embedding and, in sparse models, to refresh the support as the predicted state moves through regions where a different local linear description may be more appropriate ([Fathi et al., 2024](#)).

A one-shot rollout can fail for two related reasons. First, the learned encoder and decoder are not exact inverses, so repeated multiplication by K can push the latent state away from the region where the decoder was trained to reconstruct accurately. Second, finite-dimensional Koopman invariance is only approximate, especially near separatrices or transitions between locally valid charts. Periodic re-encoding addresses both issues by projecting the model’s own decoded prediction back through the encoder, where the sparse thresholding rule can select a support appropriate for the current predicted state rather than preserving a stale support from the initial state.

In multibasin terms, the support selected at the initial state should not be assumed to remain valid after the predicted trajectory enters another basin or another boundary-adjacent chart. Re-encoding is therefore evaluated not as teacher forcing, but as a label-free opportunity for the encoder to refresh both the continuous latent coefficients and the discrete support.

A.4 Sparse coding and LISTA

Classical sparse coding seeks a code z that reconstructs a signal from a small set of active dictionary atoms:

$$\min_z \frac{1}{2} \|x - Dz\|_2^2 + \lambda \|z\|_1, \quad (28)$$

where D is a dictionary and $\lambda > 0$ controls sparsity ([Olshausen and Field, 1996](#); [Chen et al., 1998](#); [Donoho and Elad, 2003](#)). The active set of the solution is piecewise constant over regions of the input space, while the coefficient values vary continuously within a fixed active set. This gives a union-of-subspaces geometry: each support indexes a lower-dimensional chart, and nearby supports can be grouped into a support family when threshold noise or boundary effects fragment the exact masks.

LISTA replaces an iterative sparse-coding solver with a learned finite-depth shrinkage network ([Gregor and LeCun, 2010](#)). In the notation of [equation \(8\)](#), the encoder forms a pre-code from the input and repeatedly applies learned affine refinement followed by soft thresholding. The thresholding operation is the important ingredient for this paper: it turns the latent code into both continuous coefficients and an explicit support. The experiments then ask whether that support

agrees with benchmark basin labels, whether it is functionally used by the predictor, and whether it can route local linear laws without a supplied basin label.

Observed supports also define a simple label-free graph: nodes are exact supports or merged support families, and edges are support transitions seen along rollouts. This graph view is not used as a training signal in the reported experiments, but it explains why supports are useful objects to expose. A support node can index a local predictor, a support-family node can absorb small threshold perturbations, and an observed edge can record when the representation moves from one local chart to another. The routing experiments in [table 2](#) test the one-step predictive version of this idea without assuming a known number of basins or known basin membership during training or deployment.

B Additional experimental details

Training and model rows. The fixed multibasin comparison uses the five model rows reported in [table 1](#). All are trained on length-8 observation windows for 200,000 optimization steps with minibatches of 256 windows, using the prediction, reconstruction, latent-consistency, and sparsity objective described in [section 4.5](#). The multibasin runs use AdamW, with learning rate 5×10^{-5} for the encoder and decoder, learning rate 5×10^{-6} for the latent transition, and weight decay 10^{-4} on the non-transition parameters. In the notation of [equation \(16\)](#), the relative weights are $\lambda_{\text{pred}} = 1$, $\lambda_{\text{rec}} = 0.03$, and $\lambda_{\text{lin}} = 1$; sparse rows use $\lambda_{\text{sp}} = 3 \times 10^{-3}$, while the Dense MLP row sets $\lambda_{\text{sp}} = 0$. The boundary-emphasized training distribution draws half of its resets from a pool of hard initial conditions selected without basin labels, using short probe rollouts to identify states with high perturbation sensitivity or lingering transient motion.

The saved best checkpoint is chosen by a fixed validation rule during training. Every 500 optimization steps, and again at the final step, the current model is rolled out for 200 steps from 16 held-out initial states using every-step re-encoding. The checkpoint with the lowest final validation error is saved as `checkpoint.pt`; the latest checkpoint is kept separately as `last.pt`. This validation rule is a training-side proxy for rollout stability. It does not use basin labels, test trajectories, or the post-hoc best-periodic evaluation grid.

The main Dysts comparison is used only as a forecasting stress test and includes a dense-transition LISTA row, block-diagonal LISTA rows, the Dense MLP baseline, a sparse MLP row, and block-diagonal sparse MLP rows. Dysts training trajectories are drawn from native trajectory caches with standardized coordinates, 200 cached trajectories, 30,000 stored observations per trajectory, and a 2,000-step warm-up; training samples length-10 windows from the cache training split. The cache is generated without period-based resampling. Initial conditions are based on each Dysts system’s default initial condition; the remaining cached trajectories perturb that state with Gaussian noise at scale 0.2 times the coordinate-wise Dysts standard deviation. Train, validation, and test caches use deterministic split-specific namespaces so that held-out trajectories are disjoint from training windows and reusable across model seeds. Long-horizon evaluation uses the disjoint held-out test cache, with the current paper-facing evaluation extending to 60,000 stored observations per trajectory. LISTA rows use the same learning rates as the multibasin runs, while MLP rows use learning rates 10^{-4} for encoder/decoder parameters and 10^{-5} for the latent transition. The paper-facing Dysts shortlist contains the 12 systems in [table 5](#).

The Dysts systems have system-specific native integration intervals, so the cached sequences should be interpreted as benchmark observation sequences rather than as trajectories normalized to a common physical time. Coordinates are standardized before training and evaluation. The models never observe the vector field, the continuous solver, or the native integration substeps; they receive

Table 5: Dysts forecasting shortlist. These systems are used only for long-horizon forecasting stress tests because basin labels are not available.

System	State dimension
Chua	3
Dadras	3
DequanLi	3
Hadley	3
LorenzCoupled	6
LuChenCheng	3
MultiChua	3
QiChen	3
Sakarya	3
SanUmSrisuchinwong	3
ShimizuMorioka	3
WangSun	3

uniformly spaced stored states and are evaluated in stored-step units. This is why the Dysts table reports horizons such as $H5000$ and $H60000$ rather than a shared physical-time horizon.

Recipe-selection provenance. Earlier exploratory sweeps varied LISTA depth, shrinkage scale, sparsity coefficient, sign-split encodings, transition structure, and MLP sparsity controls. Those pilots used shorter budgets or broader system lists, so their numeric grids are not treated as final protocol here. The paper-facing appendix instead reports the fixed model rows, training budgets, validation rule, held-out splits, and statistical units used for the final claims. In particular, staged recipe-discovery sweeps with 10,000-step, three-seed cells and one-step training windows were used only to choose candidate recipes. They are not pooled with the final 200,000-step, held-out-split evaluations and should not be interpreted as paper evidence.

Evaluation splits. All reported forecasting and interpretability results are evaluated on held-out trajectories, not on the training windows. Multibasin forecasting uses 100 held-out initial conditions for each system and seed. Routing fits local predictors on 256 disjoint held-out trajectories of length 256 and evaluates them on a separate set of 128 held-out rollouts. The Dysts long-horizon table uses 100 held-out cached test trajectories for each system and seed. Basin labels, when available in the procedural benchmark, are used only to define evaluation slices, score support–basin agreement, construct controlled transfer pairs, and compute statistical summaries.

Periodic forecasting. All horizons and re-encoding periods are reported in stored observation steps, not in equal physical time across systems. The no-reencoding rollout applies the learned latent transition for the full horizon before decoding. The periodic rollout decodes and re-encodes the model’s own predicted state at a fixed period. The multibasin forecasting tables select the best score over periods 10, 25, 50, and 100 stored observation steps. The Dysts long-horizon evaluation uses periods 50, 75, 100, 200, 400, 600, and 1000. These choices are fixed before aggregation and do not use the true future trajectory.

The fixed re-encoding grid is used only at evaluation time. Checkpoint selection uses the validation proxy described above, not a search over the final test-period grid, and the reported point

estimates aggregate all completed seeds rather than selecting the best seed. The best-periodic score should therefore be read as a controlled diagnostic of whether a model benefits from regular support refresh at deployment, not as teacher forcing or basin-label access.

Support diagnostics. The wrong-support ablation first constructs, for each basin in a testable system, a canonical deep-slice support from the most common exact support observed in that basin. The ablated rollout replaces the inferred support with a canonical support from a different basin and holds that replacement fixed. Table 1 reports the wrong-support/base error ratio at horizons $h = 1$ and $h = 20$; larger ratios indicate stronger functional dependence on the selected support.

Routing details. Support-routed forecasting uses fixed-size top-8 supports so every state receives a route key. The three routed predictors in table 2 are: a support-gated global transition, which masks the centered latent state by the current support before applying the learned transition; a centered support-local transition, which fits a separate ridge-regularized linear map for each fitted support; and a centered family-local transition, which uses the same local fitting procedure after merging nearby supports into support families. The ridge penalty is 10^{-4} . At evaluation time, a local route is used only if it was observed in at least 128 fitted transitions; otherwise the rollout falls back to the global latent transition. The reported routed/global ratio always compares the routed prediction with the same model’s own global rollout, not with another model family.

Support refresh. The support-refresh experiment uses top-8 supports and the two re-encoding periods reported in table 3, 1 and 10 stored observation steps. Each comparison starts from a controlled source-to-target transfer. The frozen-source rollout keeps using the source support after target entry, while the refreshed-current rollout periodically decodes, re-encodes, and re-extracts the support from its own predicted state. Basin labels are used only to construct valid transfer pairs and score post-entry routing; they are not supplied to the rollout.

Test units. For the multibasin and Dysts forecasting tables, the paired unit within each system is the training seed, and tests are performed on paired $\log_{10}$ raw mean squared errors. For support size, support-family entropy, and wrong-support ratios, the same per-system paired-seed structure is used. The support-refresh packet has one training seed, so its paired unit is the controlled transfer pair. The wrong-support ablation is reported only on systems whose deep-state subset contains at least two basins.

Compute resources. Reported multibasin and Dysts training jobs were launched as independent cluster array tasks requesting one GPU, four CPU cores, and 16GB memory per run, with a 24-hour wall-time cap. The routing and refresh diagnostics were run as CPU jobs requesting four CPU cores and 24GB memory, with a 12-hour wall-time cap. The Dysts long-horizon re-evaluation was run in 100-trajectory batches, with evaluation tasks requesting four CPU cores and 24GB memory. These values describe the requested resources for the reported packets; the wall-time caps are not measured runtimes.

C Support definitions and sensitivity

The absolute-threshold support S_{abs} marks a latent coordinate active when its magnitude exceeds 10^{-3} . It is the strict identifiability object used on the deep-state slice. The top- k support $S_{\text{top}k}$ selects the eight largest-magnitude latent coordinates. It is the deployment-like object because

every state receives a route key without a magnitude threshold. The relative-threshold support S_{rel} marks a coordinate active when its magnitude is at least 10% of the largest latent magnitude for that state. It removes global scale effects and was useful for centered local-law diagnostics, but it is too fragmented as an exact support router and is therefore retained only as a sensitivity object. Support families are produced by greedy Jaccard merging at threshold 0.5. In our results, S_{abs} saturates the conditional entropy $H(B | S)$ on the deep slice for all sparse rows and is therefore not a discriminator on its own; $H(S | B)$ and U_{exact} are the discriminative columns there. S_{top8} is the only support definition for which the dense MLP control’s failure on routing cannot be explained by route-availability artifacts.

D Multibasin benchmark inventory

Table 6 lists the 17 multibasin systems used in this paper, their basin counts available at evaluation, and the main-text display item that uses each system.

Table 6: The 17 multibasin benchmark systems. Geometry types: *multiwell* = multistable potential systems; *gated* = gated local-linear systems with explicit transitions between regions; *polygonal* = arrangement of point attractors at vertices of a polygon; *checkerboard* = grid-based potential; *cascade* = depth-cascade variants. Basin counts are revealed only at evaluation.

System	Geometry	Basins	Used in
multiwell_strong_transition	multiwell	4	Tab. 1, Fig. 5
gated_local_linear	gated	4	Tab. 1, Tab. 2
gated_transfer_linear	gated	4	Tab. 1, Tab. 3
arrested_spiral	polygonal	4	Tab. 2
cal_asymmetric_3	polygonal	3	Tab. 1
cal_high_cross_3	polygonal	3	Tab. 1
cal_hexagon_6	polygonal	6	Tab. 1
cal_octagon_8	polygonal	8	Tab. 1
cal_pentagon_5	polygonal	5	Tab. 1
cal_square_4	polygonal	4	Tab. 1
checkerboard_potential	checkerboard	9	Tab. 1
duffing_triple_well	multiwell	3	Tab. 1, Fig. 5
snic_multi	polygonal	3	Tab. 1
transition_routes_4	gated	4	Tab. 1
var_depth_gradient_4	cascade	4	Tab. 1
var_diamond_4	polygonal	4	Tab. 1
var_l_shape_5	polygonal	5	Tab. 1

E Per-seed distributions

Table 1 reports cross-system summaries of per-system IQM-over-seeds. The corresponding per-(system, seed) distributions are shown below. Figure 5 shows the paper-facing strip plot for the matched boundary-emphasized rows. Figure 6 plots per-(system, seed) histograms of forecasting MSE at every paper-facing horizon. Figure 7 plots per-(system, seed) histograms of $H(B | S)$, $H(S | B)$, and U_{exact} on the deep-state slice. Figure 8 plots the same per-seed distributions for the Dysts long-horizon benchmark. The distributions are heavy-tailed in log-MSE; we therefore report the cross-system $\log_{10}$ -IQR alongside the IQM in table 1. The cross-system $\log_{10}$ -IQR is ≤ 1.97 for every row at every horizon (the highest values come from the rows whose worst-system seeds blow up at $H1000$, dominated by `multiwell_strong_transition`), and the per-system rank ordering of rows is stable across horizons.

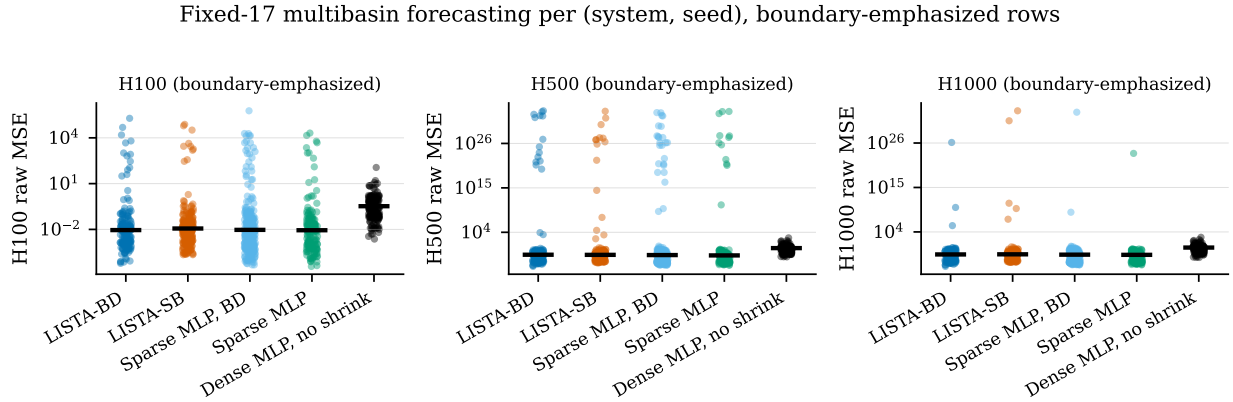


Figure 5: Per-(system, seed) forecasting MSE at $H100$, $H500$, and $H1000$ on the multibasin benchmark. Black bars are IQM values. The Dense MLP has a much heavier upper tail at every horizon.

F Full per-system tables

Tables 7–9 render the per-(system, candidate) effect sizes and Holm-corrected Wilcoxon p -values that underlie the K/N counts in table 1 and the forest plot in figure 9. Each cell gives the per-system median paired $\log_{10}$ -MSE difference (candidate minus matched-sampling Dense MLP, no-shrink baseline) over the 10 training seeds and the corresponding Holm-corrected one-sided paired Wilcoxon p -value (Holm correction across the 17 systems within each horizon-and-candidate column). Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate. Systems are sorted by the median across candidates of the per-system effect; the same ordering is used in all three tables for visual cross-reference.

The raw per-(system, seed) values, per-system bootstrap 95% CIs of the paired median, and the corresponding paired t -test p -values are released in the supplementary materials. The supplementary CSVs include per-system records, the row-level K/N summary, and the cross-system table used in the main text; the manifest lists the names and the source forecasting CSVs.

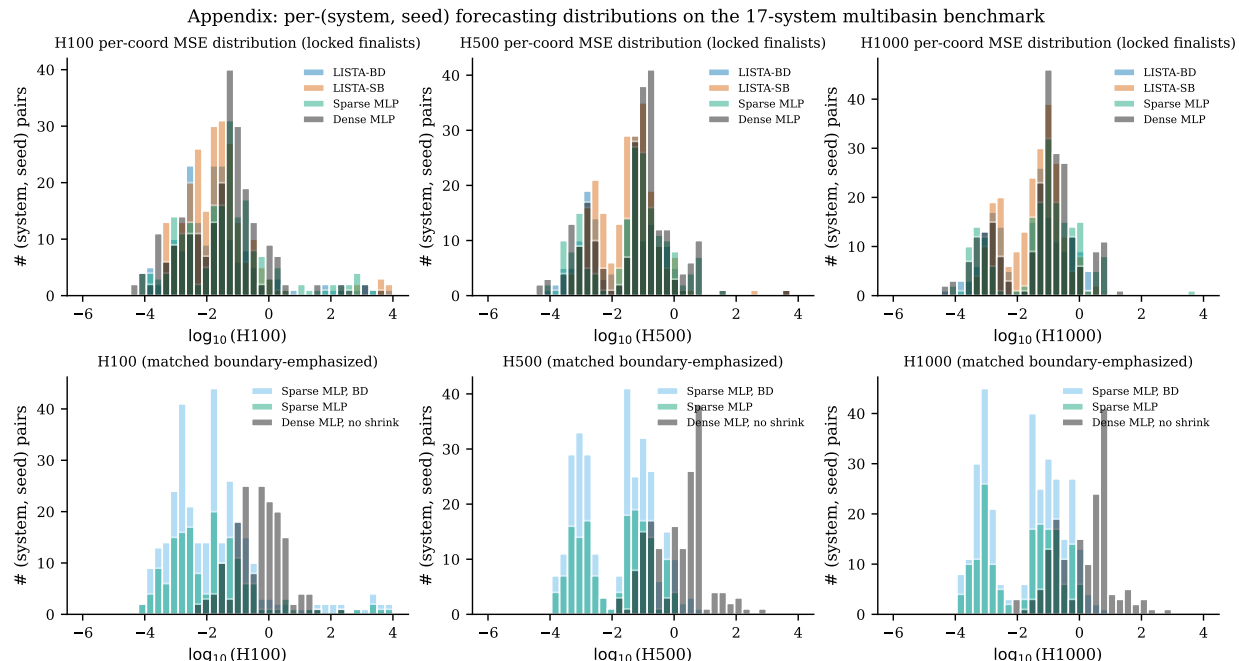


Figure 6: Per-(system, seed) forecasting MSE distributions on the multibasin benchmark. Each row is a horizon ($H100$, $H500$, $H1000$); the top row shows the LISTA finalists with the standard-sampling MLP controls (for completeness), and the bottom row shows the matched boundary-emphasized MLP controls used in the main-text comparison. Histograms are over the $\log_{10}$ raw MSE.

Per-system audit of the interpretability columns

Why the wrong-support denominator is 13 and not 17. The wrong-support ablation builds, for each system, a dictionary mapping each basin to its canonical support mask – the most-common exact support among the deep-slice states of that basin. The ablated rollout draws a mask from a basin different from the state’s own basin and holds that wrong mask fixed, so the test requires at least two distinct deep-slice canonical masks. Four of the 17 benchmark systems – `arrested_spiral` (5 basins), `cal_asymmetric_3` (3), `duffing_triple_well` (3), `var_1_shape_5` (5) – have multiple basins by design but a deep slice (top quartile by basin-depth margin) concentrated in a single basin: one basin’s attractor has a much larger margin to the second-nearest centroid than the others, so the global top-quartile filter selects only that basin’s states. This holds across all 7 trained model rows and all 9 support-threshold definitions, confirming it is a property of the system’s depth-margin geometry rather than of any encoder. The wrong-support ablation is undefined on those four systems by construction, and the corresponding K/N denominator in [table 1](#) and [tables 12](#) and [13](#) is 13 instead of 17. The four omitted systems still appear in the per-system tables with “—” entries. A queued robustness re-evaluation under a per-basin top-quartile depth criterion – which admits each basin’s relative-deep states and is expected to restore 17/17 coverage on these four systems – is documented in `docs/PER_BASIN_DEEP_SLICE_PLAN.md`; its results will appear here as a separate appendix table once the re-eval completes, and the global-slice numbers in [table 1](#) and the per-system tables below are the source-of-truth in the meantime.

Tables [10–13](#) render the full per-(system, candidate) breakdown of the four interpretability columns of [table 1](#). Each cell gives the per-system median paired difference between the candidate and the matched-sampling Dense MLP no-shrink baseline (over the 10 training seeds), and the

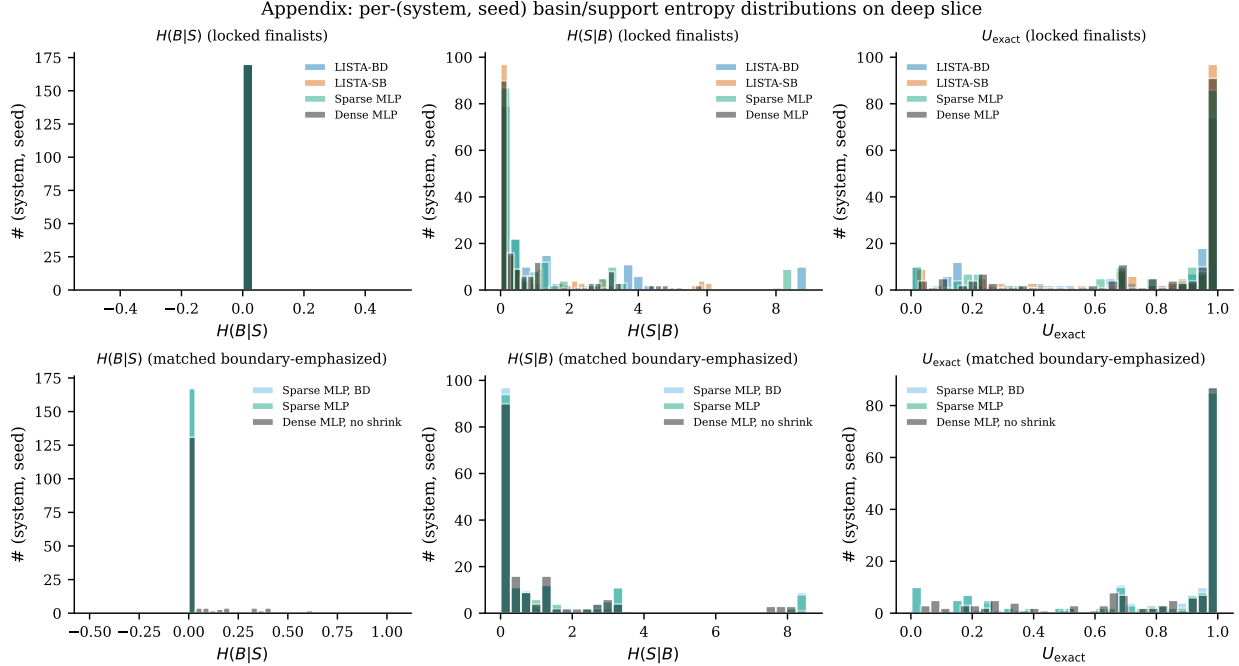


Figure 7: Per-(system, seed) histograms of basin/support agreement metrics on the deep-state slice (S_{abs}). Top row: LISTA finalists with standard-sampling MLP controls. Bottom row: matched boundary-emphasized MLP controls used in the main-text comparison. Lower is better for $H(B | S)$ and $H(S | B)$; higher is better for U_{exact} .

corresponding Holm-corrected one-sided paired Wilcoxon p -value (Holm correction across the 17 systems within each candidate column). Bold entries indicate Holm-corrected $p < 0.05$ in favor of the candidate. The alternative direction is "candidate < baseline" for $|S|$ and $H(B | F)$ (smaller is better) and "candidate > baseline" for the wrong-support ablation ratios (larger is better because the support is more functionally meaningful when the wrong one hurts more).

G Full per-system Dysts tables

The per-(system, seed) Dysts MSE values are released as `table3_dysts.csv` (cross-system summaries) and as `forecasting_rows.csv` for both Dysts evaluation packets. Cross-system $\log_{10}$ -MSE standard deviations range from 0.18 (LISTA-D at $H30000$) to 0.77 (LISTA-D at $H5000$); none exceed unity. The dense MLP control is consistent with these but has a wider per-system spread at the longest horizons (figure 4 and figure 8).

H Falsification: oracle, random, and latent-cluster controls

The routing experiments include three falsification controls that we summarise here. (i) Oracle-conditioned A_{basin} fits beat the global K on 93–100% of evaluated deep rows for every model family, including the dense no-sparsity MLP. This says local centred laws exist for many representations once a good partition is supplied; the sparse-latent contribution is that the model itself supplies an inspectable, label-free partition that does this work. (ii) Random count-matched partitions and latent k -means clusters do not reproduce the routed/global ratios in table 2: support-conditioned

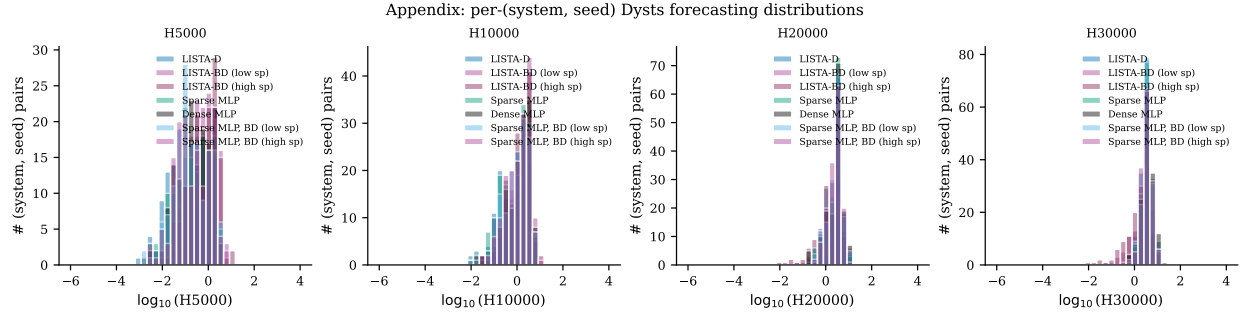


Figure 8: Per-(system, seed) forecasting MSE distributions on the Dysts long-horizon benchmark.

routing improves on these controls by an order of magnitude on the deep slice for the sparse-latent rows. (iii) On the local-Jacobian recovery diagnostic, LISTA support families often beat random count-matched partitions near attractors but the dense no-sparsity MLP can have lower absolute projected-Jacobian error because its representation is closer to the identity map. We therefore do not claim recovery of true Jacobian eigendirections; we claim support-conditioned local prediction and support-label agreement.

References

- Omri Azencot, N. Benjamin Erichson, Vanessa Lin, and Michael Mahoney. Forecasting Sequential Data Using Consistent Koopman Autoencoders. In *Proceedings of the 37th International Conference on Machine Learning*, pages 475–485. PMLR, November 2020.
- Steven L. Brunton, Bingni W. Brunton, Joshua L. Proctor, and J. Nathan Kutz. Koopman Invariant Subspaces and Finite Linear Representations of Nonlinear Dynamical Systems for Control. *PLOS ONE*, 11(2), February 2016. ISSN 1932-6203. doi: 10.1371/journal.pone.0150171. URL <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0150171>.
- Steven L. Brunton, Marko Budišić, Eurika Kaiser, and J. Nathan Kutz. Modern Koopman Theory for Dynamical Systems. *SIAM Review*, 64(2):229–340, May 2022. ISSN 0036-1445. doi: 10.1137/21M1401243. URL <https://doi.org/10.1137/21M1401243>.
- Scott Shaobing Chen, David L. Donoho, and Michael A. Saunders. Atomic Decomposition by Basis Pursuit. *SIAM Journal on Scientific Computing*, 20(1):33–61, 1998. doi: 10.1137/S1064827596304010. URL <https://doi.org/10.1137/S1064827596304010>.
- David L. Donoho and Michael Elad. Optimally sparse representation in general (nonorthogonal) dictionaries via ℓ^1 minimization. *Proceedings of the National Academy of Sciences*, 100(5): 2197–2202, March 2003. doi: 10.1073/pnas.0437847100. URL <https://doi.org/10.1073/pnas.0437847100>.
- Mahan Fathi, Clement Gehring, Jonathan Pilault, David Kanaa, Pierre-Luc Bacon, and Ross Goroshin. Course correcting koopman representations. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=A18gWgc5mi>.
- William Gilpin. Chaos as an interpretable benchmark for forecasting and data-driven modelling. In *Advances in Neural Information Processing Systems (NeurIPS) 2021*, 2021. URL <http://arxiv.org/abs/2110.05266>.

Per-system paired Wilcoxon effect sizes (10 seeds/system, Holm-corrected at $\alpha = 0.05$ across 17 systems)

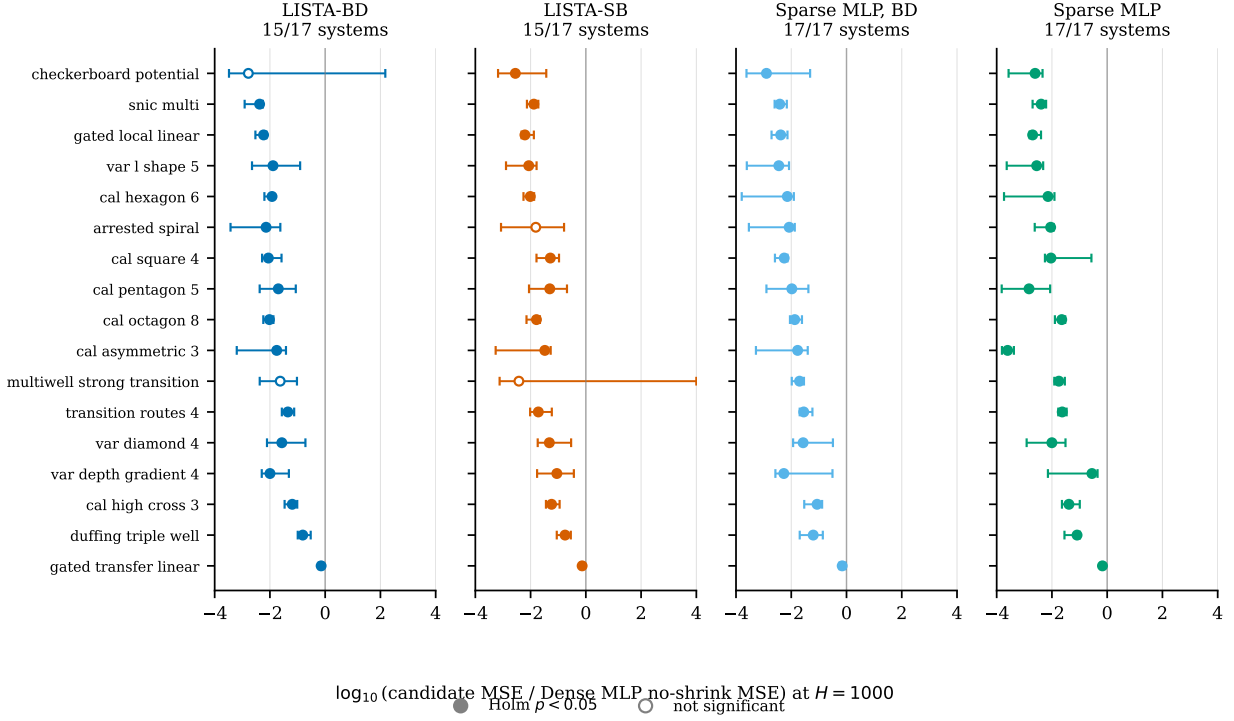


Figure 9: Per-system effect sizes at $H=1000$ versus the Dense MLP baseline. Each point is one benchmark system; the x -coordinate is the per-system median paired $\log_{10}$ -MSE difference (candidate minus baseline) over 10 seeds, with whiskers showing the 95% bootstrap interval. Filled markers clear Holm-corrected $\alpha=0.05$. Numerical values are in table 9.

William Gilpin. Model scale versus domain knowledge in statistical forecasting of chaotic systems. *Physical Review Research*, 5(4):043252, December 2023. doi: 10.1103/PhysRevResearch.5.043252. URL <https://link.aps.org/doi/10.1103/PhysRevResearch.5.043252>.

Karol Gregor and Yann LeCun. Learning Fast Approximations of Sparse Coding. In *Proceedings of the 27th International Conference on Machine Learning, ICML'10*, pages 399–406, Haifa, Israel, 2010. Omnipress. ISBN 978-1-60558-907-7. doi: 10.5555/3104322.3104374. URL <https://dl.acm.org/doi/abs/10.5555/3104322.3104374>.

D. M. Grobman. Homeomorphism of systems of differential equations. *Doklady Akademii Nauk SSSR*, 128:880–881, 1959.

Lars Grüne and Jürgen Pannek. *Nonlinear Model Predictive Control*. Communications and Control Engineering. Springer International Publishing, Cham, 2017. ISBN 978-3-319-46023-9 978-3-319-46024-6. doi: 10.1007/978-3-319-46024-6. URL <http://link.springer.com/10.1007/978-3-319-46024-6>.

Philip Hartman. A lemma in the theory of structural stability of differential equations. *Proceedings of the American Mathematical Society*, 11(4):610–620, 1960. doi: 10.1090/S0002-9939-1960-0121542-7.

Table 7: Per-system effect sizes and Holm-corrected Wilcoxon p -values at $H = 100$ on the 17-system multibasin benchmark, against the matched-sampling Dense MLP, no-shrink baseline. Δ is the per-system median paired $\log_{10}$ -MSE difference (candidate – baseline) over the 10 seeds; p_H is the one-sided paired Wilcoxon signed-rank p -value, Holm-corrected across the 17 systems within each candidate column. Bold Δ : Holm-corrected $p < 0.05$ in favor of the candidate.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
checkerboard_potential	−0.14	0.922	−0.75	0.131	+0.33	1.000	−0.75	0.193
snic_multi	−2.11	0.017	−1.62	0.017	−2.22	0.018	−2.22	0.017
gated_local_linear	−2.15	0.017	−2.10	0.017	−2.32	0.017	−2.50	0.017
var_l_shape_5	−1.86	0.017	−1.72	0.017	−1.77	0.018	−2.04	0.017
cal_hexagon_6	−1.70	0.017	−1.83	0.017	−2.07	0.018	−1.96	0.018
arrested_spiral	−2.00	0.017	−1.17	0.029	−1.90	0.018	−1.83	0.018
cal_square_4	−1.68	0.017	−1.09	0.017	−1.85	0.017	−1.29	0.017
cal_pentagon_5	−1.33	0.017	−0.95	0.017	−1.56	0.017	−2.13	0.017
cal_octagon_8	−1.96	0.017	−1.89	0.017	−1.80	0.017	−1.67	0.017
cal_asymmetric_3	−1.82	0.017	−1.49	0.017	−1.70	0.018	−2.43	0.018
multiwell_strong_transition	+2.61	1.000	+1.52	0.990	+2.52	1.000	+2.34	0.999
transition_routes_4	−1.57	0.017	−2.20	0.017	−2.23	0.017	−2.41	0.017
var_diamond_4	−1.85	0.017	−1.71	0.017	−1.52	0.017	−1.80	0.018
var_depth_gradient_4	−1.54	0.017	−0.92	0.017	−1.31	0.018	−0.79	0.018
cal_high_cross_3	−1.23	0.017	−1.14	0.017	−1.16	0.017	−1.28	0.017
duffing_triple_well	−0.54	0.027	−0.60	0.017	−1.00	0.018	−0.98	0.018
gated_transfer_linear	−0.22	0.126	−0.28	0.017	−0.36	0.017	−0.25	0.126
$K/17$ Holm $p < 0.05$	$K=14 / N=17$		$K=15 / N=17$		$K=15 / N=17$		$K=14 / N=17$	

R.E. Kalman. On the general theory of control systems. *1st International IFAC Congress on Automatic and Remote Control, Moscow, USSR, 1960*, 1(1):491–502, August 1960. ISSN 1474-6670. doi: 10.1016/S1474-6670(17)70094-8. URL <https://www.sciencedirect.com/science/article/pii/S1474667017700948>.

B. O. Koopman. Hamiltonian Systems and Transformation in Hilbert Space. *Proceedings of the National Academy of Sciences*, 17(5):315–318, May 1931. doi: 10.1073/pnas.17.5.315. URL <https://www.pnas.org/doi/10.1073/pnas.17.5.315>.

B. O. Koopman and J. v. Neumann. Dynamical Systems of Continuous Spectra. *Proceedings of the National Academy of Sciences*, 18(3):255–263, March 1932. doi: 10.1073/pnas.18.3.255. URL <https://www.pnas.org/doi/abs/10.1073/pnas.18.3.255>.

Milan Korda and Igor Mezić. Linear predictors for nonlinear dynamical systems: Koopman operator meets model predictive control. *Automatica*, 93:149–160, July 2018. ISSN 00051098. doi: 10.1016/j.automatica.2018.03.046. URL <http://arxiv.org/abs/1611.03537>.

Matthew D. Kvalheim and Shai Revzen. Existence and uniqueness of global Koopman eigenfunctions for stable fixed points and periodic orbits. *Physica D: Nonlinear Phenomena*,

Table 8: Per-system effect sizes and Holm-corrected Wilcoxon p -values at $H = 500$ on the 17-system multibasin benchmark. Format identical to table 7.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
checkerboard_potential	+6.46	1.000	+1.15	0.647	+2.83	1.000	-0.58	0.084
snic_multi	-2.46	0.017	-1.87	0.017	-2.38	0.017	-2.45	0.017
gated_local_linear	-2.26	0.017	-2.11	0.017	-2.29	0.017	-2.56	0.017
var_l_shape_5	-1.89	0.017	-2.21	0.017	-2.42	0.017	-2.76	0.017
cal_hexagon_6	-1.94	0.017	-2.13	0.017	-2.45	0.017	-2.33	0.017
arrested_spiral	-2.43	0.017	+3.03	0.647	-2.31	0.017	-2.16	0.017
cal_square_4	-1.92	0.017	-1.32	0.017	-2.26	0.017	-1.49	0.017
cal_pentagon_5	-1.71	0.017	-1.27	0.017	-1.98	0.017	-2.73	0.017
cal_octagon_8	-2.15	0.017	-2.01	0.017	-1.91	0.017	-1.76	0.017
cal_asymmetric_3	-1.98	0.017	-1.92	0.017	-2.27	0.017	-3.29	0.017
multiwell_strong_transition	+23.90	1.000	+16.90	0.997	+24.96	1.000	+24.51	1.000
transition_routes_4	-1.56	0.017	-2.11	0.017	-1.59	0.017	-1.88	0.017
var_diamond_4	-1.55	0.017	-1.26	0.017	-1.45	0.017	-2.06	0.017
var_depth_gradient_4	-1.81	0.017	-1.05	0.017	-1.56	0.017	-0.92	0.017
cal_high_cross_3	-1.17	0.017	-1.14	0.017	-1.15	0.017	-1.32	0.017
duffing_triple_well	-0.88	0.017	-0.83	0.017	-1.32	0.017	-1.24	0.017
gated_transfer_linear	+1.85	0.158	-0.16	0.017	-0.16	0.017	-0.19	0.017
$K/17$ Holm $p < 0.05$	$K=14 / N=17$		$K=14 / N=17$		$K=15 / N=17$		$K=15 / N=17$	

425:132959, November 2021. ISSN 0167-2789. doi: 10.1016/j.physd.2021.132959. URL <https://www.sciencedirect.com/science/article/pii/S0167278921001160>.

Yueheng Lan and Igor Mezić. Linearization in the large of nonlinear systems and Koopman operator spectrum. *Physica D: Nonlinear Phenomena*, 242(1):42–53, January 2013. ISSN 0167-2789. doi: 10.1016/j.physd.2012.08.017.

Scott Linderman, Matthew Johnson, Andrew Miller, Ryan Adams, David Blei, and Liam Paninski. Bayesian Learning and Inference in Recurrent Switching Linear Dynamical Systems. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, pages 914–922. PMLR, April 2017. URL <https://proceedings.mlr.press/v54/linderman17a.html>.

Bethany Lusch, J. Nathan Kutz, and Steven L. Brunton. Deep learning for universal linear embeddings of nonlinear dynamics. *Nature Communications*, 9(1):4950, November 2018. ISSN 2041-1723. doi: 10.1038/s41467-018-07210-0.

Giorgos Mamakoukas, Maria Castano, Xiaobo Tan, and Todd Murphey. Local Koopman Operators for Data-Driven Control of Robotic Systems. In *Robotics: Science and Systems XV*. Robotics: Science and Systems Foundation, June 2019. ISBN 978-0-9923747-5-4. doi: 10.15607/RSS.2019.XV.054. URL <http://www.roboticsproceedings.org/rss15/p54.pdf>.

D. Q. Mayne, J. B. Rawlings, C. V. Rao, and P. O. M. Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6):789–814, June 2000. ISSN 0005-1098. doi:

Table 9: Per-system effect sizes and Holm-corrected Wilcoxon p -values at $H = 1000$ on the 17-system multibasin benchmark. This is the numerical version of the forest plot in figure 9. Format identical to table 7.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
checkerboard_potential	−2.78	0.216	− 2.55	0.017	− 2.90	0.017	− 2.61	0.017
snic_multi	− 2.37	0.017	− 1.88	0.017	− 2.42	0.017	− 2.39	0.017
gated_local_linear	− 2.23	0.017	− 2.21	0.017	− 2.38	0.017	− 2.70	0.017
var_l_shape_5	− 1.89	0.017	− 2.07	0.017	− 2.45	0.017	− 2.55	0.017
cal_hexagon_6	− 1.92	0.017	− 2.01	0.017	− 2.14	0.017	− 2.14	0.017
arrested_spiral	− 2.14	0.017	−1.81	0.129	− 2.08	0.017	− 2.05	0.017
cal_square_4	− 2.06	0.017	− 1.28	0.017	− 2.26	0.017	− 2.04	0.017
cal_pentagon_5	− 1.70	0.017	− 1.31	0.017	− 1.98	0.017	− 2.83	0.017
cal_octagon_8	− 2.02	0.017	− 1.79	0.017	− 1.87	0.017	− 1.65	0.017
cal_asymmetric_3	− 1.75	0.017	− 1.48	0.017	− 1.77	0.017	− 3.60	0.017
multiwell_strong_transition	−1.63	0.084	−2.42	0.500	− 1.70	0.042	− 1.75	0.042
transition_routes_4	− 1.35	0.017	− 1.72	0.017	− 1.55	0.017	− 1.63	0.017
var_diamond_4	− 1.57	0.017	− 1.32	0.017	− 1.57	0.017	− 2.00	0.017
var_depth_gradient_4	− 2.00	0.017	− 1.05	0.017	− 2.27	0.017	− 0.55	0.017
cal_high_cross_3	− 1.19	0.017	− 1.24	0.017	− 1.07	0.017	− 1.38	0.017
duffing_triple_well	− 0.81	0.017	− 0.75	0.017	− 1.21	0.017	− 1.09	0.017
gated_transfer_linear	− 0.15	0.017	− 0.13	0.017	− 0.16	0.017	− 0.17	0.017
$K/17$ Holm $p < 0.05$	$K=15 / N=17$	$K=15 / N=17$	$K=15 / N=17$	$K=17 / N=17$	$K=17 / N=17$	$K=17 / N=17$	$K=17 / N=17$	$K=17 / N=17$

10.1016/S0005-1098(99)00214-9. URL <https://www.sciencedirect.com/science/article/pii/S0005109899002149>.

Igor Mezić. Spectral properties of dynamical systems, model reduction and decompositions. *Nonlinear Dynamics*, 41(1–3):309–325, 2005. doi: 10.1007/s11071-005-2824-x.

Bruno A. Olshausen and David J. Field. Emergence of simple-cell receptive field properties by learning a sparse code for natural images. *Nature*, 381(6583):607–609, June 1996. ISSN 1476-4687. doi: 10.1038/381607a0. URL <https://doi.org/10.1038/381607a0>.

Samuel E. Otto and Clarence W. Rowley. Linearly recurrent autoencoder networks for learning dynamics. *SIAM Journal on Applied Dynamical Systems*, 18(1):558–593, 2019. doi: 10.1137/18M1177846.

Shaowu Pan and Karthik Duraisamy. On the lifting and reconstruction of nonlinear systems with multiple invariant sets. *Nonlinear Dynamics*, 112(12):10157–10165, June 2024. ISSN 1573-269X. doi: 10.1007/s11071-024-09581-0. URL <https://doi.org/10.1007/s11071-024-09581-0>.

Clarence W. Rowley, Igor Mezić, Shervin Bagheri, Philipp Schlatter, and Dan S. Henningson. Spectral analysis of nonlinear flows. *Journal of Fluid Mechanics*, 641:115–127, 2009. doi: 10.1017/S0022112009992059.

Table 10: Per-system effect sizes and Holm-corrected Wilcoxon p -values for $|S|$ (mean active-coordinate count on the deep slice) on the 17-system multibasin benchmark. Alternative: candidate $|S| < \text{baseline } |S|$. Lower (more negative) is better for the candidate. Latent dimension is $d_z=64$ for LISTA-SB and $d_z=256$ for LISTA-BD and the MLP rows.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
snic_multi	−174.91	0.017	−233.11	0.017	−171.26	0.017	−171.16	0.017
gated_local_linear	−167.78	0.017	−229.59	0.017	−160.43	0.017	−160.28	0.017
cal_asymmetric_3	−158.00	0.017	−229.02	0.017	−159.00	0.017	−162.00	0.017
gated_transfer_linear	−154.26	0.017	−227.24	0.017	−157.98	0.017	−158.41	0.017
transition_routes_4	−162.97	0.017	−228.84	0.017	−152.28	0.017	−152.68	0.017
duffing_triple_well	−163.37	0.017	−230.04	0.017	−149.16	0.017	−149.16	0.017
var_diamond_4	−147.30	0.017	−226.48	0.017	−155.50	0.017	−155.61	0.017
arrested_spiral	−149.91	0.017	−225.92	0.017	−154.04	0.017	−154.19	0.017
cal_pentagon_5	−156.14	0.017	−225.99	0.017	−142.92	0.017	−142.86	0.017
multiwell_strong_transition	−137.88	0.017	−225.31	0.017	−149.39	0.017	−149.34	0.017
cal_hexagon_6	−153.12	0.017	−227.34	0.017	−143.44	0.017	−143.44	0.017
var_l_shape_5	−151.37	0.017	−227.92	0.017	−143.60	0.017	−143.42	0.017
cal_square_4	−152.42	0.017	−225.23	0.017	−141.23	0.017	−141.23	0.017
var_depth_gradient_4	−150.24	0.017	−225.29	0.017	−140.58	0.017	−140.24	0.017
cal_octagon_8	−146.49	0.017	−226.48	0.017	−143.17	0.017	−143.17	0.017
checkerboard_potential	−143.32	0.017	−221.69	0.017	−138.58	0.017	−138.58	0.017
cal_high_cross_3	−144.04	0.017	−226.00	0.017	−134.51	0.017	−135.50	0.017
$K/17$ Holm $p < 0.05$	$K=17 / N=17$		$K=17 / N=17$		$K=17 / N=17$		$K=17 / N=17$	

Peter J. Schmid. Dynamic mode decomposition of numerical and experimental data. *Journal of Fluid Mechanics*, 656:5–28, 2010. ISSN 0022-1120. doi: 10.1017/S0022112010001217.

Haojie Shi and Max Q.-H. Meng. Deep Koopman Operator With Control for Nonlinear Systems. *IEEE Robotics and Automation Letters*, 7(3):7700–7707, July 2022. ISSN 2377-3766. doi: 10.1109/LRA.2022.3184036.

Naoya Takeishi, Yoshinobu Kawahara, and Takehisa Yairi. Learning koopman invariant subspaces for dynamic mode decomposition. In *Advances in Neural Information Processing Systems 30*, volume 30, 2017. URL <https://papers.nips.cc/paper/6713-learning-koopman-invariant-subspaces-for-dynamic-mode-decomposition>.

F.D. Torrisi and A. Bemporad. Hysdel-a tool for generating computational hybrid models for analysis and synthesis problems. *IEEE Transactions on Control Systems Technology*, 12(2): 235–249, 2004. doi: 10.1109/TCST.2004.824309.

Jonathan H. Tu, Clarence W. Rowley, Dirk M. Luchtenburg, Steven L. Brunton, and J. Nathan Kutz. On dynamic mode decomposition: Theory and applications. *Journal of Computational Dynamics*, 1(2):391–421, December 2014. ISSN 2158-2491. doi: 10.3934/jcd.2014.1.391. URL <https://www.aims sciences.org/en/article/doi/10.3934/jcd.2014.1.391>.

Table 11: Per-system effect sizes and Holm-corrected Wilcoxon p -values for $H(B \mid F)$ (basin uncertainty given the support *family*, deep slice). Alternative: candidate $H(B \mid F) < \text{baseline}$. Lower is better for the candidate.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
transition_routes_4	-1.38	0.017	-1.38	0.017	-1.38	0.017	-1.38	0.017
cal_square_4	-1.37	0.017	-0.86	0.017	-1.37	0.017	-1.37	0.017
var_diamond_4	-1.37	0.017	-1.22	0.017	-1.37	0.017	-1.37	0.020
multiwell_strong_transition	-1.26	0.017	-1.23	0.017	-1.29	0.017	-1.29	0.017
cal_octagon_8	-1.34	0.017	-1.09	0.017	-1.09	0.017	-1.09	0.017
snic_multi	-1.09	0.017	-1.09	0.017	-1.09	0.018	-1.09	0.017
gated_local_linear	-1.07	0.017	-1.07	0.017	-1.07	0.017	-1.07	0.017
gated_transfer_linear	-1.06	0.017	-1.06	0.017	-1.06	0.017	-1.06	0.017
cal_hexagon_6	-0.92	0.017	-0.66	0.017	-0.77	0.018	-0.79	0.020
cal_pentagon_5	-0.64	0.023	+0.00	0.375	-0.64	0.023	-0.64	0.027
var_depth_gradient_4	-0.62	0.017	-0.31	0.027	-0.62	0.018	-0.62	0.027
checkerboard_potential	-1.24	0.017	-0.54	0.017	-0.14	0.017	-0.13	0.020
cal_high_cross_3	-0.01	0.023	+0.00	0.375	-0.01	0.078	-0.01	0.039
cal_asymmetric_3	+0.00	1.000	+0.00	1.000	+0.00	1.000	+0.00	1.000
duffing_triple_well	+0.00	1.000	+0.00	1.000	+0.00	1.000	+0.00	1.000
arrested_spiral	+0.00	1.000	+0.00	1.000	+0.00	1.000	+0.00	1.000
var_l_shape_5	+0.00	1.000	+0.00	1.000	+0.00	1.000	+0.00	1.000
$K/17$ Holm $p < 0.05$	$K=13 / N=17$		$K=11 / N=17$		$K=12 / N=17$		$K=13 / N=17$	

Matthew O. Williams, Ioannis G. Kevrekidis, and Clarence W. Rowley. A Data-Driven Approximation of the Koopman Operator: Extending Dynamic Mode Decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, December 2015. ISSN 1432-1467. doi: 10.1007/s00332-015-9258-5. URL <https://doi.org/10.1007/s00332-015-9258-5>.

Matthew O. Williams, Clarence W. Rowley, and Ioannis G. Kevrekidis. A kernel-based method for data-driven koopman spectral analysis. *Journal of Computational Dynamics*, 2(2):247–265, May 2016. ISSN 2158-2491. doi: 10.3934/jcd.2015005. URL <https://www.aims sciences.org/article/doi/10.3934/jcd.2015005>.

Table 12: Per-system effect sizes and Holm-corrected Wilcoxon p -values for the wrong-support-over-base MSE ratio at $h=1$ (deep slice). Alternative: candidate ratio $>$ baseline. Higher is better for the candidate (using the wrong support hurts the prediction more, indicating the model genuinely uses the support).

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
multiwell_strong_transition	+497.20	0.013	+828.73	0.013	+1134.12	0.013	+1477.16	0.013
var_depth_gradient_4	+4851.48	0.013	+802.44	0.013	+1086.62	0.013	+1121.45	0.013
cal_pentagon_5	+4857.91	0.013	+636.10	0.013	+1679.06	0.013	+1388.96	0.013
cal_high_cross_3	+1359.69	0.013	+1082.86	0.013	+2653.52	0.013	+3301.41	0.013
checkerboard_potential	+1161.51	0.013	+731.70	0.013	+3083.50	0.013	+3577.67	0.013
cal_octagon_8	+6579.32	0.013	+3159.16	0.013	+3563.95	0.013	+3563.95	0.013
cal_square_4	+15284.66	0.013	+2200.14	0.013	+5580.87	0.013	+5249.74	0.013
var_diamond_4	+5602.16	0.013	+4745.27	0.013	+7947.22	0.013	+10718.65	0.013
gated_transfer_linear	+6274.01	0.013	+3317.50	0.013	+8587.81	0.013	+7348.76	0.013
cal_hexagon_6	+7829.62	0.013	+5473.99	0.013	+12232.14	0.013	+11641.91	0.013
transition_routes_4	+108283.56	0.013	+21622.27	0.013	+35090.81	0.013	+26428.24	0.013
snic_multi	+20622.20	0.013	+12107.59	0.013	+125008.24	0.013	+407479.13	0.013
gated_local_linear	+74134.78	0.013	+65733.62	0.013	+425426.24	0.013	+350747.37	0.013
$K/17$ Holm $p < 0.05$	$K=13 / N=13$		$K=13 / N=13$		$K=13 / N=13$		$K=13 / N=13$	

Table 13: Per-system effect sizes and Holm-corrected Wilcoxon p -values for the wrong-support-over-base MSE ratio at $h=20$. Alternative: candidate $>$ baseline. Higher is better.

System	LISTA-BD		LISTA-SB		Sparse MLP, BD		Sparse MLP	
	Δ	p_H	Δ	p_H	Δ	p_H	Δ	p_H
multiwell_strong_transition	-0.61	0.652	-0.43	0.770	-0.64	0.968	-0.71	0.993
cal_octagon_8	+4.51	0.013	+3.76	0.021	+11.49	0.013	+5.76	0.013
cal_pentagon_5	+11.60	0.013	+3.19	0.048	+7.42	0.013	+6.30	0.013
cal_square_4	+7.09	0.013	+0.56	0.387	+8.20	0.013	+9.21	0.013
var_depth_gradient_4	+1.83	0.158	-0.17	0.770	+18.37	0.013	+19.03	0.021
cal_high_cross_3	+0.32	0.432	+0.91	0.387	+22.91	0.014	+36.89	0.021
checkerboard_potential	+3.99	0.020	+11.00	0.026	+27.07	0.013	+41.31	0.013
gated_transfer_linear	+25.19	0.013	+8.56	0.122	+43.78	0.013	+20.78	0.013
cal_hexagon_6	+15.98	0.013	+0.76	0.117	+45.83	0.013	+45.83	0.013
var_diamond_4	+25.89	0.013	+4.07	0.026	+69.31	0.013	+55.66	0.013
transition_routes_4	+139.16	0.013	+17.63	0.021	+217.00	0.013	+149.91	0.013
snic_multi	+390.23	0.013	+45.01	0.013	+473.34	0.013	+627.79	0.013
gated_local_linear	+834.09	0.013	+1763.95	0.013	+6860.15	0.013	+5992.93	0.013
$K/17$ Holm $p < 0.05$	$K=10 / N=13$		$K=7 / N=13$		$K=12 / N=13$		$K=12 / N=13$	